

第6章 AT89 系列单片机的内部资源及应用

AT89 系列单片机是低功耗、高性能 CMOS 8 位单片机，具有在系统可编程 Flash 存储器，使用 Atmel 公司高密度非易失性存储器技术制造，与 MCS-51 系列单片机产品指令和引脚完全兼容，AT89 系列单片机内部基本功能模块有：8 位的并行口 P0、P1、P2、P3，可编程的中断系统，16 位的定时/计数器 T0、T1，全双工的异步串行口 UART。

本章主要介绍 AT89 系列单片机内部基本功能模块及其编程应用。

6.1 AT89 系列单片机的并行口及其应用

单片机内部有 4 个 8 位的并行口 P0 ~ P3，应用非常广泛，可以作为输出口，直接连接输出设备（如发光二极管），也可以作为输入口，直接连接输入设备（如开关设备）。由于单片机应用场合的多样性，单片机并行口除了可以外接指示灯、开关外，还可以外接继电器、显示器、蜂鸣器、电动机、打印机、键盘等常见的 I/O 设备。

1. 指示灯

在单片机应用系统中，常会用到指示灯。洗衣机、电饭煲、空调等家用电器中都有指示灯指示电器的当前工作状态。本节中讲述单片机并行口控制指示灯的控制操作。

【例 6-1】 用 51 单片机的并行口 P1 口驱动 8 个发光二极管，使 8 个发光二极管从上到下轮流点亮。

硬件电路如图 6-1 所示。当 P1.0 上输出低电平 0 时，相应的发光二极管 LED0 有电流通过发光，当输出电平为高电平 1 时，发光二极管两端无电压差，无电流通过不发光。要使得 LED0 ~ LED7 轮流点亮，则应该让 P1 口分别输出数据 FEH、FDH、FBH、F7H、EFH、DFH、BFH、7FH，为防止数据变化太快，LED 灯轮流点亮的频率过快，人眼看不清 LED 灯的状态变化，当从一个状态变换到另一个状态时，需要调用延时程序延时一段时间。

(1) 汇编语言程序

```
MOV A,#0FEH
AGAIN:MOV P1,A
LCALL DELAY
RL A
SJMP AGAIN
DELAY:MOV R7,#20
MOV R6,#250
DJNZ R6,$
DJNZ R7,DELAY
RET
```

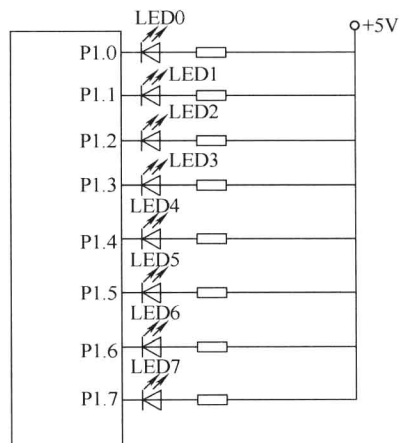


图 6-1 流水灯接口电路

(2) C51 语言程序

1) 查表法:

```
#include <reg52.h>

Void main( )
{
    unsigned char LED[8] = {0xfe,0xfd,0xfb,0xf7,0xef,0xdf,0xbf,0x7f};
    unsigned char i;
    unsigned int j;
    while{1}
    {
        for(i=0;i<8;i++)
        {
            P1 = LED[i];
            for(j=0;j<30000;j++) ; //延时
            if (i==8) i=0xff; //为了下一次循环 LED 灯从上到下点亮,置 i 为 ffH
        }
    }
}
```

上述例子中给出了一种流水灯的闪烁样式。实际应用中,只需要改变表中数据,指示灯的点亮方式就可以改变。

除了用查表的方式给 P1 口输出数据,还可以根据灯点亮的状态变化规律,采取循环移位的方式设置流水灯。

2) 移位法:

```
#include <reg52.h>
#include <intrins.h>
void main( )
{
    unsigned char i,a=0xfe;
    unsigned int j;
    while{1}
    {
        P1 = a;
        a = _crol_(a,1); //调用循环左移函数_crol_,变量 a 循环左移 1 位
        for(j=0;j<30000;j++) ; //延时
    }
}
```

2. 蜂鸣器

在许多智能家电和工业控制中都有蜂鸣器,它以发声的形式将机器状态告知人们。图 6-2 为一种蜂鸣器的接口电路,工作原理是:当 P1.0 引脚输出 0 时,晶体管导通,在蜂鸣器两端加工作电压,蜂鸣器发出声音;当 P1.0 引脚输出 1 时,晶体管截止,蜂鸣器不发声。通过给 P1.0 引脚上输出高低电平持续时间不同的信号,可以使得蜂鸣器发出不同频率的声音,也可以应用此原理播放音乐。

【例 6-2】 编程使蜂鸣器响铃 10 次,每次响铃发出

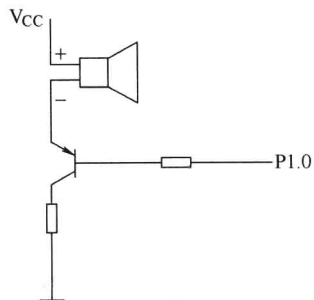


图 6-2 蜂鸣器接口电路

250Hz 的声音，持续时间为 0.5s，静音 1s，单片机使用 12MHz 的晶振。

实现该功能的程序如下：

```

MOV R3,#10
AA1: CLR P1.0
MOV R1,#250
AA2: LCALL DELAY2           ;调用延时 2ms 子程序
CPL P1.0
DJNZ R1,AA2                 ;250Hz 响铃 0.5s
SETB P1.0
MOV R2,#2
AA3: MOV R1,#250
AA4: LCALL DELAY2
DJNZ R1,AA4
DJNZ R2,AA3                 ;静音 1s
DJNZ R3,AA1
RET
DELAY2:MOV R7,#10           ;延时 2ms 程序
AA5: MOV R6,#100
DJNZ R6,$                   ;双周期指令,执行一次耗时 2μs
DJNZ R7,AA5
RET

```

6.2 AT89 系列单片机的中断系统

中断 (Interrupt) 是日常生活中常见的现象。比如，我们正在工作时，突然来了一件更为紧急的任务，我们必须终止当前的工作，而去处理突发事件，处理完以后再继续之前的工作。在这个事件中，突发事件中断了我们正常的工作——中断请求，我们处理突发事件——中断响应，处理完之后继续工作——中断返回。

实际上，现代计算机都具有对外界突发事件作出及时处理的功能，这需要依靠计算机内部的中断系统来实现。例如，在计算机工业控制系统中，当设备发生故障、系统内部出错以及用户要求一些特殊的功能和操作时，必须要求计算机暂停常规工作，转而去处理这些突发事件，待处理完毕后，继续执行原来的程序。可见，中断系统对单片机处理突然事件具有非常重要的意义。本节重点讲述单片机的中断系统的概况和中断系统的使用方法。

6.2.1 中断的基本概念

计算机系统中止当前的正常工作，转入处理突发事件，待突发事件处理完毕后，再回到原来被中断的地方，继续原来的工作，这样的整个过程称为中断。能够实现这种功能的部件就称为中断系统 (Interrupt System)。产生中断请求的事件称为中断源 (Interrupt Source)。中断源向 CPU 发出请求信号称为中断请求 (Interrupt Request)。CPU 中止当前工作而处理中断称为中断响应 (Interrupt Response)。可见，在中断系统中必须先有中断源请求中断，再

由中断系统做出反应（响应中断或者不响应中断）。中断过程如图 6-3 所示。

一般计算机系统允许有多个中断源，当几个中断源同时向 CPU 请求中断时，CPU 优先响应哪一个中断请求源，一般根据中断源的排队，优先处理排在队伍前面的中断请求。当 CPU 正在处理一个中断请求时，又发生了另一个优先级比它高的中断源请求，CPU 能够暂时中止执行对原来中断源的处理程序，转而去处理优先级更高的中断请求，待处理完以后，再继续执行原来低级中断处理程序，这样的过程称为二级中断嵌套。二级中断嵌套的中断过程如图 6-4 所示。

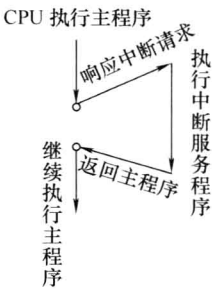


图 6-3 中断响应过程

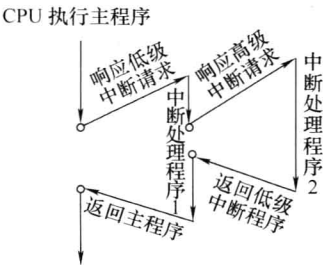


图 6-4 二级中断嵌套过程

6.2.2 AT89 系列单片机的中断系统

AT89 系列单片机的中断系统结构随型号不同有所不同，包括中断源数目、中断优先级、中断控制寄存器等都有差别。例如，典型产品 89S52 单片机中有 6 个中断源，具有 2 个中断优先级，可以实现二级中断嵌套。每一个中断源可以设置为高优先级或者低优先级，用户可以根据需要来设定 CPU 是否开放中断，每个中断源允许或禁止向 CPU 请求中断。89S52 的中断系统结构如图 6-5 所示。

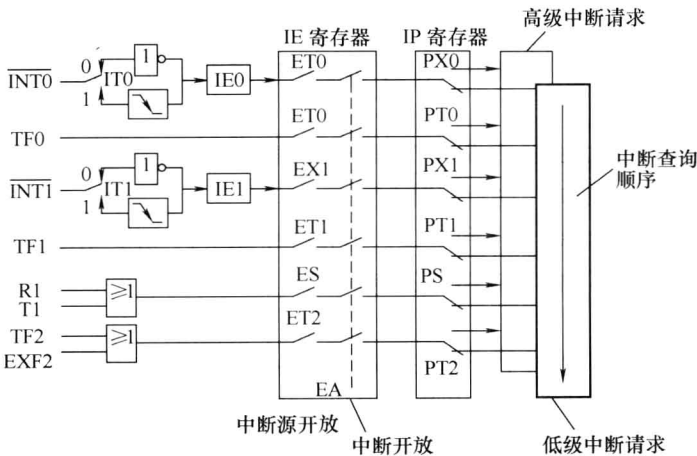


图 6-5 89S52 中断系统结构

1. 89S52 中断源

89S52 有 6 个中断源：2 个外部事情中断请求源（ $\overline{\text{INT0}}$ 、 $\overline{\text{INT1}}$ ）和 4 个内部中断源（分

别为定时/计数器 T0、T1、T2 计数溢出事情中断请求和串行口发送或者接受完一个字节数据中断请求源)。89S51 有 5 个中断源, 没有 T2 计数溢出中断。

(1) 外部中断源

外部事件中断由单片机外部的信号触发, 外中断 0 和外中断 1 的中断请求信号分别由单片机的 $\overline{\text{INT0}}$ (P3.2)、 $\overline{\text{INT1}}$ (P3.3) 引脚输入有效的中断请求信号。

外中断 0 和外中断 1 的中断标志位和它们的触发方式控制位在特殊功能寄存器——定时器控制寄存器 TCON 中的低 4 位如图 6-6 所示。

D7	D6	D5	D4	D3	D2	D1	D0
TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0

图 6-6 定时器控制寄存器 TCON

IT0: 外部中断 0 触发方式控制位。

IT0 = 0, 外部中断 0 触发方式选择为电平触发方式。这种方式中, $\overline{\text{INT0}}$ 端输入低电平时, 为有效的中断请求信号, 置位 IE0。CPU 在每一个机器周期采样 $\overline{\text{INT0}}$ (P3.2) 引脚的输入电平。当采样到低电平时置“1”IE0; 当采样到高电平时, 清“0”IE0。采用电平触发方式时, 外部中断源信号必须保持低电平有效, 直到该中断被 CPU 响应, 同时在该中断服务程序执行完之前, 外部中断源信号必须清除; 否则, 将产生另一次中断请求。

IT0 = 1, 外部中断 0 触发方式选择为边沿触发方式。这种触发方式 CPU 在每一个机器周期采样 $\overline{\text{INT0}}$ (P3.2) 引脚的输入电平。如果前一次机器周期中采样到 $\overline{\text{INT0}}$ (P3.2) 引脚为高电平, 接着下一个机器周期采样到 $\overline{\text{INT0}}$ (P3.2) 引脚为低电平, 即 $\overline{\text{INT0}}$ (P3.2) 引脚电平产生负跳变, 则置“1”IE0。IE0 为 1 时, 向 CPU 申请中断, 直到该中断被 CPU 响应, 由硬件完成 IE0 清 0。由于每个机器周期采样一次外部中断的输入电平。因此, 采用边沿触发方式, 外部中断源输入的高电平和低电平时间必须保持两个机器周期以上, 才能保证 CPU 检查到电平的跳变。

IE0: 外中断 0 的中断请求标志位。当外部中断源 $\overline{\text{INT0}}$ (P3.2) 引脚上有有效的中断请求信号, 则置位 IE0, 向 CPU 请求中断, 当 CPU 响应该中断时由硬件清“0”IE0 (边沿触发方式)。

IT1: 外部中断 1 触发方式选择位。IT1 = 0, 外中断 1 为电平触发方式; IT1 = 1, 外中断 1 为边沿触发方式。其功能和 IT0 完全相同。

IE1: 外中断 1 的中断请求标志位。IE1 = 1, 外中断 1 向 CPU 请求中断, CPU 响应中断请求后, 由硬件清“0”IE1 位 (边沿触发方式)。

(2) 内部中断源

特殊功能寄存器 TCON 中与中断相关的还有两位, 分别为 TF0 和 TF1 位。

TF0: 定时/计数器 T0 计数溢出中断标志位。T0 允许计数后, 从计数初值开始加 1 计数, 当计数计满后 (计数器所有位均为“1”), 再加 1, 则计数溢出, 此时由硬件自动置“1”TF0, 向 CPU 请求中断, 一直保持到 CPU 响应该中断时才由内部硬件清“0”。

TF1: 定时/计数器 T1 计数溢出中断标志位。T1 允许计数后, 从计数初值开始加 1 计数, 当计数溢出时, 硬件自动置“1”TF1, 向 CPU 请求中断, 一直到 CPU 响应中断时由硬件清“0”。

89S52 内部中断源除了 T0 和 T1 外, 还有定时/计数器 T2 和串行口中断。

定时/计数器 T2 中断：定时/计数器 T2 的计数溢出标志位 TF2 为“1”，或者 T2 的外部中断标志位 EXF2 为“1”，作为有效的中断请求，向 CPU 请求中断。CPU 响应中断时不能自动清 0，必须由软件清 0。

串行口中断：串行口的接受中断标志 RI (SCON.0) 和发送中断标志 TI (SCON.1) 逻辑或以后作为内部的一个中断源。当串行口发送缓冲器发送完一个字符数据后，由硬件自动把发送中断标志位 TI 置“1”，向 CPU 请求中断。需要注意的是，CPU 响应中断后，发送中断标志位 TI 不会自动清“0”，必须由用户在中断处理程序中用软件清“0”。

2. 中断控制

(1) 中断使能控制

AT89 系列单片机的中断允许控制通过 2 级控制，第 1 级由 6 个中断源各自的中断允许位确定屏蔽或允许某个中断源的中断请求，第 2 级通过 1 个总控制位来确定 CPU 开放或者屏蔽中断请求。AT89 系列单片机通过一个特殊功能寄存器 IE——中断允许寄存器来保存这些中断允许控制位，IE 寄存器的字节地址为 0A8H，寄存器各位的位地址为 0A8H ~ 0AFH，其格式如图 6-7 所示。

D7	D6	D5	D4	D3	D2	D1	D0
EA	—	ET2	ES	ET1	EX1	ET0	EX0

图 6-7 中断允许寄存器 IE

EA：AT89S 系列单片机的 CPU 中断允许控制位。EA = 1，CPU 中断允许，允许中断源向 CPU 申请中断，即 CPU 开放中断；EA = 0，CPU 中断屏蔽，即禁止中断源向 CPU 申请中断，CPU 屏蔽所有的中断。

EX0：外中断 0 的中断允许位。EX0 = 1，中断允许，允许外中断 0 向 CPU 申请中断；EX0 = 0，中断屏蔽，禁止外中断 0 的中断请求。

ET0：定时/计数器 T0 的中断允许位。ET0 = 1，中断允许，即允许定时/计数器 T0 计数溢出时向 CPU 申请中断；ET0 = 0，禁止中断，即禁止定时/计数器 T0 计数溢出时申请中断。

EX1：外中断 1 的中断允许位。EX1 = 1，外中断 1 中断允许；EX1 = 0，外中断 1 中断屏蔽。

ET1：定时/计数器 T1 的中断允许位。ET1 = 1，定时/计数器 T1 中断允许；ET1 = 0，定时/计数器 T1 中断屏蔽。

ES：串行口中断允许位。ES = 1，串行口中断允许；ES = 0，串行口中断屏蔽。

ET2：定时/计数器 T2 的中断允许位。ET2 = 1，定时/计数器 T2 中断允许；ET2 = 0，定时/计数器 T2 中断屏蔽。

(2) 中断优先级控制

AT89 系列单片机有两个中断优先级，每一个中断源可以设定其中断优先级别为高优先级中断或者低优先级中断，可以实现二级中断嵌套。

对于多个中断源请求中断时，处理中断请求的原则是：

1) 多个中断源同时向 CPU 请求中断时，优先响应高优先级中断源的中断请求。

2) 当 CPU 正在处理低级中断服务程序，可以被高优先级的中断请求所中断，但不能被另外一个同级的中断请求中断，也不能被其他低优先级级别的中断请求中断。若 CPU 正在执

行高优先级的中断服务程序，则不能被任何中断源所中断，一直执行到结束，执行完中断返回指令 RETI，返回主程序再执行一条指令后，才能响应新的中断源请求。

AT89 系列单片机中断系统中通过特殊功能寄存器 IP 实现 6 个中断源的中断优先级别的控制，其字节地址为 0B8H，寄存器中各位的位地址为 0BFH ~ 0B8H，格式如图 6-8。

D7	D6	D5	D4	D3	D2	D1	D0
—	—	PT2	PS	PT1	PX1	PT0	PX0

图 6-8 中断优先级寄存器 IP

PX0：外部中断 0 的中断优先级控制位。PX0 = 1，外部中断 0 定义为高优先级中断；PX0 = 0，外中断 0 定义为低优先级中断。

PT0：定时/计数器 T0 中断优先级控制位。PT0 = 1，定时/计数器 T0 中断定义为高优先级中断；PT0 = 0，定时/计数器 T0 中断定义为低优先级中断。

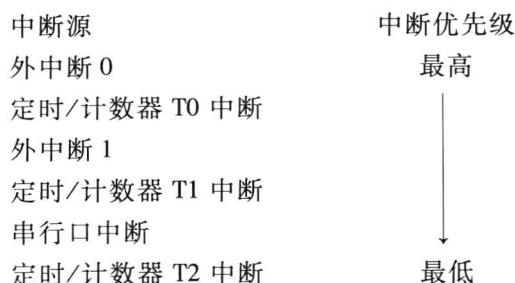
PX1：外部中断 1 的中断优先级控制位。PX1 = 1，外部中断 1 定义为高优先级中断；PX1 = 0，外中断 1 定义为低优先级中断。

PT1：定时/计数器 T1 中断优先级控制位。PT1 = 1，定时/计数器 T1 中断定义为高优先级中断；PT1 = 0，定时/计数器 T1 中断定义为低优先级中断。

PS：串行口中断优先级控制位。PS = 1，串行口中断定义为高优先级中断；PS = 0，串行口中断定义为低优先级中断。

PT2：定时/计数器 T2 中断优先级控制位。PT2 = 1，定时/计数器 T2 中断定义为高优先级中断；PT2 = 0，定时/计数器 T2 中断定义为低优先级中断。

在同一级别的中断源请求源中，一个内部的硬件查询序列确定优先响应哪一个中断请求。这样便在同级优先级中，由查询顺序确定中断响应的先后顺序。89S52 中断查询的顺序如下：



CPU 在响应中断时，先查询外中断 0 是否有中断请求，若有中断请求，响应其中断请求，不再查询其他同级的中断请求。若没有中断请求，接着查询定时器 T0 的中断请求。以此类推，外中断 1、定时/计数器 T1 中断、串行口中断、定时/计数器 T2 中断。

AT89 系列单片机复位以后，特殊功能寄存器 IE、IP 的内容均为 0，由初始化程序对 IE、IP 编程，从而编程控制中断系统是否开放中断、允许哪些中断源申请中断以及中断源的中断优先级。

3. 中断响应过程

在初始化程序中设置 CPU 中断允许和中断允许控制位为中断允许状态，当中断源有有效的中断请求信号时，相应的中断标志位置“1”，CPU 在每一个机器周期顺序检查每一个

中断源，并按优先级处理所有中断请求，在下一个机器周期响应最高级的中断请求。

(1) 响应一级中断请求的条件

因此，CPU 能够响应一级中断请求时应具备以下条件：

- 1) 来自中断源有效的中断请求。
- 2) CPU 中断允许。
- 3) 这个中断源的中断允许位处于允许状态。
- 4) CPU 刚刚结束一条指令的执行。

CPU 响应中断时，必须在一条指令执行结束之后，另外，CPU 执行 RETI 指令和对寄存器 IE 和 IP 访问指令时，即使指令执行结束也不会立即响应中断，必须再执行一条指令才会响应中断请求。

(2) 响应二级中断嵌套请求的条件

AT89 系列单片机可以有二级中断嵌套，若 CPU 能够响应二级中断嵌套请求时应具备以下条件：

- 1) 来自中断源有效的中断请求。
- 2) CPU 中断允许。
- 3) 这个中断源的中断允许位处于允许状态。
- 4) 这个中断源的中断优先级为高级。
- 5) CPU 刚刚结束主程序或者低级中断服务程序中一条指令的执行。

在二级中断嵌套中，若 CPU 正在处理一个中断服务程序，此时又来了一个新的中断请求，向 CPU 申请中断，如果新的中断请求的优先级高于正在处理的中断源的优先级，则 CPU 立即停止正在处理的中断，转去响应新的中断请求，待新的中断请求处理完毕后，继续处理原来的中断，原来中断服务程序也处理完后，返回主程序，形成了二级中断嵌套的情形。如果新的中断请求的优先级不高于正在处理的中断源，则 CPU 继续处理正在处理的中断服务程序，待中断处理结束返回主程序后，再响应新的中断请求。

CPU 一旦响应中断请求，立即产生一个硬件调用，使程序转移到相应的中断服务程序的入口地址处去执行下面的中断服务程序，直到执行完中断服务程序的最后一条中断返回指令 RETI，返回被中断的程序。

AT89 系列单片机各个中断源的中断服务程序的入口地址是固定的，具体如表 6-1 所示。

表 6-1 中断源与入口地址

中断源	入口地址
外部中断 0	0003H
定时器 T0	000BH
外部中断 1	0013H
定时器 T1	001BH
串行口中断	0023H
定时器 T2	002BH

通常，在中断入口处，执行一条跳转指令，跳到用户设计的中断处理程序入口。CPU 执行中断处理程序一直到 RETI 指令为止，RETI 指令是中断服务程序的最后一条指令，CPU

执行这条指令后，清“0”响应中断时所置位的优先级状态触发器，然后从堆栈中弹出栈顶的 2 字节到程序计数器 PC，堆栈中保存的数据为执行中断服务程序前，CPU 正在执行指令的下一条指令的地址，当这个地址装入程序计数器 PC 后，CPU 将接着执行被中断的程序。用户编写中断服务程序时，最后一条指令必须为 RETI 指令，用以返回被中断的程序，恢复中断现场。

6.2.3 外部事件中断及应用

AT89S 系列单片机提供两个外部事件中断源。外部事件中断请求信号由 $\overline{\text{INT0}}$ （P3.2 引脚）或者 $\overline{\text{INT1}}$ （P3.3 引脚）输入单片机的中断系统，中断触发方式可以由 IT0 或者 IT1 位选择为电平触发或者边沿触发方式。电平触发方式中低电平信号为有效的中断请求信号，当单片机外部中断输入引脚上为低电平信号时，触发中断，中断系统把相应的中断标志位置“1”，发出中断请求。需要注意的是，中断请求的低电平必须保持到 CPU 响应该中断为止，并且必须在本次中断返回前变为高电平，撤出中断请求信号，以免产生多次中断请求。若外部中断触发方式选择为边沿触发，则外部中断引脚上出现电平负跳变时触发中断，中断系统在连续的两个机器周期中，前一个机器周期检测到 $\overline{\text{INT0}}$ 引脚或者 $\overline{\text{INT1}}$ 引脚为高电平，在后一个机器周期检测到低电平，则置位响应的中断请求标志位 IE0 或者 IE1，发出中断请求，高电平和低电平至少各自保持 1 个机器周期，这样保证可以被正确检测到。CPU 响应外部中断 0 的中断请求后，自动将中断请求标志 IE0 或者 IE1 清 0，撤除本次中断请求。

1. 外部事件中断源的初始化

1) 设置外部事件中断请求信号的触发方式。如果外部中断触发方式采用电平触发方式，IT0 或者 IT1 位清 0，用指令 CLR IT0 或者 CLR IT1，由于单片机复位后，IT0 和 IT1 被清 0，默认为电平触发方式。因此，有些程序中省略这一步。如果用边沿触发方式，IT0 或者 IT1 位置“1”，用指令 SETB IT0 或者 SETB IT1。

2) 开放 CPU 中断允许位：SETB EA。

3) 设置外部事件中断允许控制位：SETB EX0 或者 SETB EX1。

4) 设置中断源中断优先级。

由于单片机复位后，特殊功能寄存器 IP 中各位为 0，若没有中断优先级嵌套，所有中断源的中断优先级都为同一级别——低级，程序中不需要设置中断优先级。如果有中断嵌套，则需要设置有些中断源中断优先级为高级，有些中断源中断优先级为低级，中断优先级别高的中断源优先响应。例如，指令 SETB PX0，设置外部中断 0 中断优先级为高级；指令 CLR PX1，设置外部中断 1 中断优先级为低级。由于单片机复位后 IP 中断优先级控制寄存器被清 0，默认所有的中断源中断优先级为低级，所以很多程序中省略低优先级的中断控制。

主程序中，对中断系统初始化编程时，也可以采用字节操作指令完成中断允许控制和中断优先级控制，格式如下：

```
MOV IE, #DATA1    //中断允许控制
MOV IP, #DATA2     //中断优先级控制
```

例如，完成中断系统初始化编程，开放 6 个中断源的中断允许，同时设置外部中断 1 中断优先级为高级。

```
MOV IE, #0BFH
```

```
MOV IP, #04H
```

2. 外部中断应用举例

【例 6-3】 如图 6-9 所示, P1.0 ~ P1.7 为输出线, 外接指示灯 L0 ~ L7, 采用外部中断 0 电平触发方式改变指示灯 L0 ~ L7 的显示状态。正常显示时, 灯 L0 ~ L7 自上而下逐一点亮, 当有外部中断请求时, 灯 L0 ~ L7 全部点亮并闪烁显示 10 次。闪烁完成后, 继续从暂停的位置接着逐个点亮灯的操作。

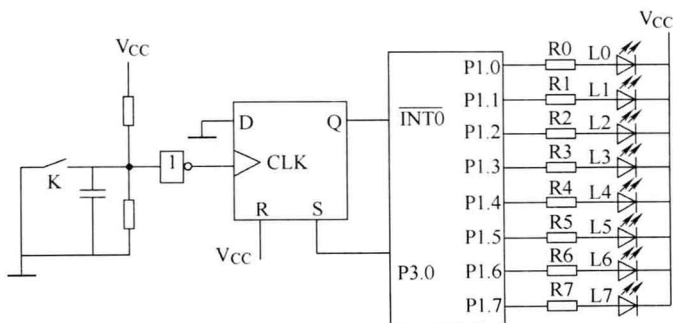


图 6-9 电平触发方式中断系统图

在图 6-9 中, 当按键 K 每按动一次, 产生一个正脉冲, 接入 D 触发器 CLK 端, 则 $Q = D$, Q 端输出低电平并锁存。Q 端和单片机的 INT0 (P3.2 引脚) 相接, 以低电平触发方式向 CPU 申请中断, CPU 响应中断后, 由单片机 P3.0 输出“0”至 D 触发器 S 端, 此时 D 触发器工作在置位状态, Q 端输出“1”, 从而撤销单片机外部中断 0 引脚上的低电平中断请求信号。这样, 下一次按动按键 K 时, 可以触发一次新的中断。

主程序:

```
ORG 0000H
LJMP MAIN
ORG 0003H
LJMP INT0
ORG 0030H
MAIN: MOV SP, #70H           ;设置堆栈区
      CLR ITO                 ;电平触发方式
      SETB EA                 ;开放 CPU 中断允许
      SETB EX0                ;设置外中断 0 中断允许
      CLR PX0                 ;中断系统只有一个中断源, 设置为低中断优先级
      MOV A, #0FEH
DISPLAY: MOV P1, A           ;给 P1 口送数据, 输出显示
        ACALL DELAY          ;延时
        RL A                  ;左移, 产生下一个显示控制码
        AJMP DISPLAY
DELAY:  MOV R7, #200          ;延时子程序
        MOV R6, #100
        DJNZ R6, $
```

```
DJNZ R7,DELO
```

```
RET
```

中断服务程序:

```
ORG 3000H
INT0: PUSH ACC                ;保护现场
      PUSH PSW
      CLR P3.0
      MOV R5,#10
REPEAT: MOV A,#00H            ;全部点亮
        MOV P1,A
        ACALL DELAY
        MOV A,#0FFH          ;全部熄灭
        MOV P1,A
        ACALL DELAY
        DJNZ R5,REPEAT       ;闪烁 10 次
        POP PSW
        POP ACC
        RETI
      END
```

C51 程序:

```
#include <reg52.h>
#include <intrins.h>
void delay ( );
sbit P3_0 = P3^0;
unsigned char a;
main ( )
{
    a = 0xfe;
    IT0 = 0;                //外中断 0 电平触发方式
    EA = 1;
    EX0 = 1;
    PX0 = 0;
    while (1)
    {
        P1 = a;
        a = _crol_(a,1);    //左移,产生下一个显示控制码
        delay ( );
    }
}

void delay( )
{
    unsigned int b;
    b = 20000;
```

```

while (b>0) b--;
}

extern0 ( ) interrupt 0 using 1 //中断函数
{
    unsigned char i;
    P3_0 = 0;
    for (i = 10; i > 0; i--) //闪烁 10 次
    {
        P1 = 0x00;
        delay( );
        P1 = 0xff;
        delay( );
    }
}

```

对于电平触发方式,需要在中断返回前撤销中断请求信号,这样单片机在检测到 $\overline{\text{INT0}}$ 引脚或者 $\overline{\text{INT1}}$ 引脚上为高电平时,才能清中断请求标志位 IE0 或者 IE1。对于边沿触发方式的外部中断 0 或者外部中断 1, CPU 在响应中断后由硬件自动清除其中断标志位 IE0 或者 IE1,无须采取其他措施。

【例 6-4】 外部中断源的扩展。单片机系统中只提供了两个外部中断源,在某些单片机应用系统中有 2 个以上的中断源需要处理。此时,已有的两个外部中断源已经不能满足需要,需要扩展多个外部中断源。

图 6-10 中,共有 6 个外部中断源,分别为 0 号、1 号、2 号、3 号、4 号、5 号中断源,中断请求采用电平触发方式,0 号中断源的中断优先级别最高,单片机优先响应,单片机接收到 0 号中断源中断请求时,8 个 LED 灯闪烁显示 10 次,1 号、2 号、3 号、4 号、5 号中断源为低级中断源,当有其中一个发出中断请求信号时,则触发外部中断 $\overline{\text{INT1}}$ 中断请求,然后在 $\overline{\text{INT1}}$ 的中断服务程序中,通过查询 P1.0 ~ P1.4 的状态,判定是哪一个中断请求,然后执行响应的中断服务程序,查询的顺序,决定这几个中断源中断优先级,先查询的优先级别

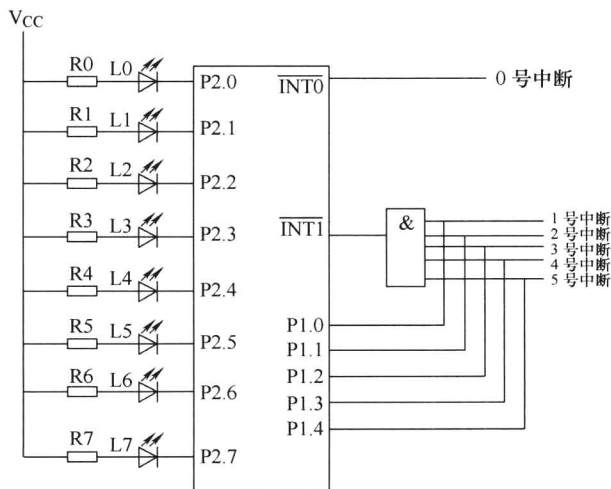


图 6-10 外部中断源扩展电路

高，后查询的优先级别低。本例中低优先级中断源的中断优先循序为 5 号、4 号、3 号、2 号、1 号。

主程序：

```

        ORG 0000H
        LJMP MAIN
        ORG 0003H           ;外中断 0 中断入口地址
        LJMP PINT0
        ORG 0013H           ;外中断 1 中断入口地址
        LJMP PINT1
        ORG 0030H
MAIN:    CLR  IT0            ;设置外中断 0 为电平触发方式
        CLR  IT1            ;设置外中断 1 为电平触发方式
        MOV  IE,#85H        ;设置 CPU 中断允许,外中断 0、外中断 1 中断允许
        MOV  IP,#01H        ;设置外中断 0 为高级,外中断 1 为低级
        LOOP:  ....
        LJMP LOOP

```

INT0 的中断服务程序：

```

PINT0:   PUSH  ACC
        PUSH  PSW
        SETB  RS1           ;把工作寄存器区切换到 2 区,中断处理程序中使用 2 区 R0 ~ R7
        MOV  R1,#10         ;闪烁显示 10 次
FLASH:   CLR  A
        MOV  P1,A           ;P1 口灯全亮
        ACALL DELAY
        CPL  A
        MOV  P1,A           ;P1 口灯全熄灭
        ACALL DELAY
        DJNZ  R1,FLASH
        POP  PSW
        POP  ACC

```

INT1 的中断服务程序：

```

PINT1:   CLR  EA            ;关中断
        PUSH  ACC
        PUSH  PSW           ;保护现场
        SETB  EA            ;开中断
        JNB  P1.4  PINT1_5
        JNB  P1.3  PINT1_4
        JNB  P1.2  PINT1_3
        JNB  P1.1  PINT1_2
        JNB  P1.0  PINT1_1
RETURN:  CLR  EA            ;关中断
        POP  PSW

```

```

        POP  ACC           ;恢复现场
        SETB  EA           ;开中断
        RETI
PINT1_5:.....           ;5 号中断服务程序
        AJMP  RETURN
PINT1_4:.....           ;4 号中断服务程序
        AJMP  RETURN
PINT1_3:.....           ;3 号中断服务程序
        AJMP  RETURN
PINT1_2:.....           ;2 号中断服务程序
        AJMP  RETURN
PINT1_1:.....           ;1 号中断服务程序
        AJMP  RETURN

```

此程序对应的 C51 程序如下:

```

#include <reg52. h>

void delay ( );           //延时函数声明
void PINT1_5();           //5 号中断调用的函数声明
void PINT1_4();           //4 号中断调用的函数声明
void PINT1_3();           //3 号中断调用的函数声明
void PINT1_2();           //2 号中断调用的函数声明
void PINT1_1();           //1 号中断调用的函数声明
sbit P1_0 = P1^0;
sbit P1_1 = P1^1;
sbit P1_2 = P1^2;
sbit P1_3 = P1^3;
sbit P1_4 = P1^4;
unsigned char a;
main ( )
{
    IT0 = 0;               //外中断 0 电平触发方式
    IE = 0x85;
    IP = 0x01;
    ...
    while (1);
}
void delay( )
{
    unsigned int b;
    b = 20000;
    while (b > 0) b -- ;
}
void PINT1_5() { ..... } //5 号中断调用的函数定义
void PINT1_4() { ..... } //4 号中断调用的函数

```

```

void PINT1_3() { ..... } //3 号中断调用的函数
void PINT1_2() { ..... } //2 号中断调用的函数
void PINT1_1() { ..... } //1 号中断调用的函数
extern0 ( ) interrupt 0 using 2 //0 号中断函数
{
    unsigned char i;
    for (i = 10; i > 0; i -- )
    {
        P1 = 0x00;
        delay( );
        P1 = 0xff;
        delay( );
    }
}

extern1 ( ) interrupt 1 //外部中断 1 中断函数
{
    if (P1_4 == 0) PINT1_5(); //1 ~ 5 号中断为同级中断,按照查询顺序只响应一个
    else if (P1_3 == 0) PINT1_4();
        else if (P1_2 == 0) PINT1_3();
            else if (P1_1 == 0) PINT1_2();
                else if (P1_0 == 0) PINT1_1();
}

```

6.3 AT89 系列单片机定时/计数器

各种型号的单片机,无论其功能强弱,都有定时/计数器,在单片机中,通常计数器和定时器设计成一个部件——计数器,当计数脉冲的周期一定时,计数器就作为定时器,定时时间就是计数器计数次数和计数脉冲周期的乘积。计数就是对计数脉冲进行计数,输入一个脉冲,计数器加 1,再输入一个脉冲,计数器再次加 1。

在智能控制系统中,经常要用到定时和计数的功能,定时/计数器特别有用,定时/计数器可以实现下列功能。

- 1) 定时操作功能。如采用定时/计数器实现定时扫描键盘、定时扫描输入口的状态;如到了预先的定时时间,启动外部的某个操作部件。
- 2) 计数操作功能。对外部输入的脉冲信号进行加法计数或测量输入信号的周期等参数。
- 3) 定时输出功能。从单片机的某个引脚定时输出高低电平,并使得高低电平保持一定的时间,例如产生占空比一定的周期脉冲信号。
- 4) 看门狗系统。当单片机系统程序跑偏时,定时器产生复位信号,重新启动系统正常工作。

AT89 系列单片机一般有 2 个或者 3 个 16 位可编程定时/计数器,用于定时或者计数。根据单片机型号不同,有的单片机内部有 2 个 16 位定时/计数器 T0、T1,有的单片机内部有 3 个 16 位定时/计数器 T0、T1、T2。

6.3.1 定时/计数器的一般结构和工作原理

定时/计数器的一般结构如图 6-11 所示, 由一个 N 位计数器、计数时钟信号、计数控制开关、计数溢出标志位等主要部分组成, 51 系列单片机中计数器的计数方式为加 1 计数, 计数时钟信号可以是单片机内部时钟信号, 也可以是外部输入的时钟信号, 当计数器计数加到计数溢出时, 由单片机内部硬件自动把 TF 计数溢出标志位置 1, 作为一个独立的中断请求源向 CPU 申请中断。

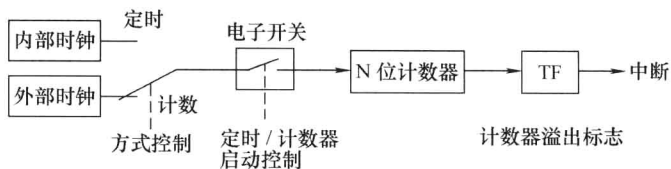


图 6-11 定时/计数器结构图

1. 定时方式

当计数器的计数脉冲为单片机内部确定周期的时钟信号时, 对此确定周期的时钟信号计数, 即实现定时。如计数时钟信号为周期 T 的脉冲信号, 从计数器中计数初值 a 开始加 1 计数, 直至计数溢出所占用的时间为

$$t = T(2^N - a)$$

其中, T 为时钟信号周期; N 为计数器的位数; a 为计数器的计数初值; t 为从计数器开始计数到计数溢出时的时间, 即定时时间。

定时/计数器工作于定时方式时, 是对单片机内部的机器周期信号进行计数, 机器周期信号由振荡信号经过 12 分频得到, 故一个机器周期为一个振荡周期的 12 倍。

例如, 某单片机应用系统, 使用 12MHz 的晶振, 定时/计数器为 16 位的定时/计数器, 计数初值为 15536, 则该定时/计数器的定时时间为多长?

单片机的机器周期 T 为

$$T = 12 \frac{1}{f_{\text{osc}}} = 12 \frac{1}{12 \times 10^6} \mu\text{s} = 1 \mu\text{s}$$

定时时间 t 为

$$t = T(2^N - a) = 1 \mu\text{s}(2^{16} - 15536) = 50\text{ms}$$

这种工作方式称为定时器方式, 其计数目的就是为了定时。例如, 当计数器从计数初值到计数溢出时, 对 P1.0 取反, 则 P1.0 输出一个方波, 高低电平持续时间是 50ms。

2. 计数方式

计数方式是对外部输入的脉冲信号计数, 外部输入的脉冲信号从特定的引脚上输入, 计数器对脉冲信号加 1 计数, 即信号发生从 1 ~ 0 的负跳变, 计数器自动加 1, 外部输入信号的高低电平的维持时间必须不小于 1 个机器周期, 因此计数输入信号的频率不能高于晶振频率的 1/24。例如, 对外部脉冲信号计数可以统计外部事件的发生次数, 在规定时间内测得外部输入脉冲数, 可以求得脉冲的平均周期。这种方式通常称为计数方式。

6.3.2 定时/计数器 T0、T1 功能和使用方法

51 系列单片机与 16 位定时/计数器 T0、T1 有关的特殊功能寄存器有 TH0、TL0、TH1、

TL1、TMOD、TCON，与定时/计数器中断相关的寄存器还有 IE、IP。

特殊功能寄存器 TH0、TL0 为 16 位定时/计数器 T0 的高 8 位和低 8 位寄存器，TH1、TL1 为 16 位定时/计数器 T1 的高 8 位和低 8 位寄存器，TMOD 为 T0 和 T1 的方式寄存器，TCON 为 T0 和 T1 的状态和控制寄存器，存放 T0、T1 的运行控制位和溢出中断标志位。通过对 TH0、TL0、TH1、TL1 的初始化编程来设定 T0、T1 计数器的计数初值，通过对 TCON 和 TMOD 的编程来选择 T0、T1 的工作方式和控制 T0、T1 的运行。

1. 方式寄存器 TMOD

特殊功能寄存器——定时/计数器工作方式寄存器 TMOD (Timer/Counter Mode Control Register)，用于设置 T0、T1 的工作方式，它的高 4 位为 T1 的方式字段，低 4 位为 T0 的方式字段，它们的含义完全相同，如图 6-12。

D7	D6	D5	D4	D3	D2	D1	D0
GATE	C/ $\overline{T}$	M1	M0	GATE	C/ $\overline{T}$	M1	M0

图 6-12 方式寄存器 TMOD

8 位中低 4 位 D0 ~ D3 用于控制定时/计数器 T0，高 4 位 D4 ~ D7 用于控制定时/计数器 T1。具体控制字含义如下：

(1) M1 和 M0 工作方式选择位

M1 和 M0 两位对应定时/计数器的工作方式，对应关系如表 6-2 所示。

表 6-2 定时/计数器工作方式选择

M1	M0	功能说明
0	0	方式 0，13 位定时/计数方式
0	1	方式 1，16 位定时/计数方式
1	0	方式 2，8 位自动重装初值定时/计数方式
1	1	方式 3，T0 分为两个独立的 8 位定时/计数器，T1 停止计数

(2) C/ $\overline{T}$ 定时/计数模式选择位

定时/计数器工作于定时还是计数方式，主要是选择的计数脉冲源的不同，计数脉冲源来自单片机内部时钟源为定时方式，计数脉冲源来自外部为计数方式，在 TMOD 中由 C/ $\overline{T}$ 来实际选择计数脉冲源，含义如下：

C/ $\overline{T}$ =0 为定时方式。在定时方式中，由振荡器输出的脉冲信号经过 12 分频信号作为计数脉冲信号，就是每一个机器周期定时/计数器加 1。定时/计数器从计数初值开始加“1”计数，到计数溢出这段时间是固定的，为定时时间，所以这种方式为定时方式。

C/ $\overline{T}$ =1 为计数方式。这种方式采用外部引脚上的输入脉冲作为计数脉冲，T0 计数器的计数脉冲由 P3.4 引脚输入，T1 计数器的计数脉冲由 P3.5 引脚输入。内部硬件电路在每个机器周期采样外部引脚的状态，当一个周期采样到高电平，下一个周期采样到低电平，计数器加 1。也就是说，在外部输入电平发生负跳变时，计数器加 1 计数。外部输入脉冲高电平和低电平时间必须保持在一个机器周期以上，故外部输入计数脉冲信号的频率最高是晶振频率的 1/24。对外部脉冲信号计数通常用于对输入的脉冲数计数，或者用于测定特定信号的周期、频率等参数。

(3) GATE 门控位

GATE = 1 时，是否启动定时/计数器开始计数，受外部引脚 $\overline{\text{INT0}}$ （或者 $\overline{\text{INT1}}$ ）上电平控制， $\overline{\text{INT0}}$ 引脚控制 T0， $\overline{\text{INT1}}$ 引脚控制 T1，定时/计数器开始计数 $\overline{\text{INT0}}$ （或者 $\overline{\text{INT1}}$ ）上必须为高电平。

GATE = 0 时，是否启动定时/计数器开始计数，不受外部引脚输入电平的控制。

2. 控制寄存器 TCON

定时/计数器控制寄存器 TCON 占用的字节地址为 88H，TCON 中的高 4 位为定时器的计数启动位和溢出标志位，低 4 位为外部中断的触发方式控制位和外部中断请求标志。TCON 的格式如图 6-13 所示。

D7	D6	D5	D4	D3	D2	D1	D0
TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0

图 6-13 定时/计数器控制寄存器 TCON 的格式

(1) T0 运行控制位 TR0

TR0 为定时/计数器 T0 的运行控制位，由软件置“1”和清“0”来启动 T0 或者停止 T0 计数。当 GATE 门控位为 0 时，T0 的计数由 TR0 控制，TR0 为 1 时启动 T0，计数开始，T0 从计数初值开始进行加 1 计数，TR0 为 0 时停止 T0 计数。

当 GATE 门控位为 1 时，TR0 为 1 且 $\overline{\text{INT0}}$ （P3.2）引脚输入高电平时 T0 才计数，TR0 为 0，或者 $\overline{\text{INT0}}$ （P3.2）引脚输入低电平，T0 都处于不工作状态，停止计数。

(2) T0 溢出标志位 TF0

当 T0 被允许计数以后，T0 从初值开始加“1”计数，计数溢出时 TF0 置“1”，同时计数器清 0。所谓计数溢出，即 T0 加“1”计数到 N 位计数器数值全为 1 时，再来一个脉冲，则计数溢出。TF0 可以由程序查询当前 TF0 的状态，TF0 也是单片机的内部中断请求源之一，若 T0 中断允许，则在 CPU 响应中断请求时，由硬件清“0”TF0。

(3) T1 运行控制位 TR1

TR1 为定时/计数器 T1 的运行控制位，由软件置“1”和清“0”来启动 T1 或者停止 T1 计数。当 GATE 门控位为 0 时，T1 的计数由 TR1 控制，TR1 为 1 时启动 T1 计数开始，T1 从计数初值开始进行加 1 计数，TR1 为 0 时停止 T1 计数。

当 GATE 门控位为 1 时，TR1 为 1 且 $\overline{\text{INT1}}$ （P3.3）引脚输入高电平时 T1 才计数，TR1 为 0，或者 $\overline{\text{INT1}}$ （P3.3）引脚输入低电平，T1 都停止计数。

(4) T1 溢出标志位 TF1

当 T1 允许计数以后，T1 从初值开始加“1”计数，计数溢出时置“1”TF1。TF1 与 TF0 一样，可以支持查询方式和中断方式。

3. T0、T1 的工作方式和工作原理

定时/计数器 T0 有 4 种工作方式，可以工作于方式 0、方式 1、方式 2、方式 3；定时/计数器 T1 有 3 种工作方式，可以工作于方式 0、方式 1、方式 2。工作方式不同，计数器在结构上有所不同，功能上也有差别，方式 0、方式 1、方式 2 定时/计数器 T0 和 T1 功能和结构完全一致，T0 工作于方式 3 时，定时/计数器分成两个独立的 8 位定时/计数器，此时借助定时/计数器 T1 的控制位和标志位，而定时/计数器 T1 停止计数。

下面以定时/计数器 T0 为例说明定时/计数器 4 种工作方式的结构和工作原理。

(1) 方式 0

当 T0 方式字段中 M1M0 = 00 时, 定时/计数器工作方式 0, 方式 0 为 13 位的定时计数方式, 定时/计数器 T0 方式 0 的逻辑结构图如图 6-14 所示。13 位计数器, 由 TL0 的低 5 位和 TH0 中的 8 位组成, TH0 中 8 位作为计数器的高 8 位, TL0 中的 5 位作为计数器的低 5 位, 当 TL0 中低 5 位计数溢出时向 TH0 进位, TH0 计数溢出时置“1”溢出标志 TF0。

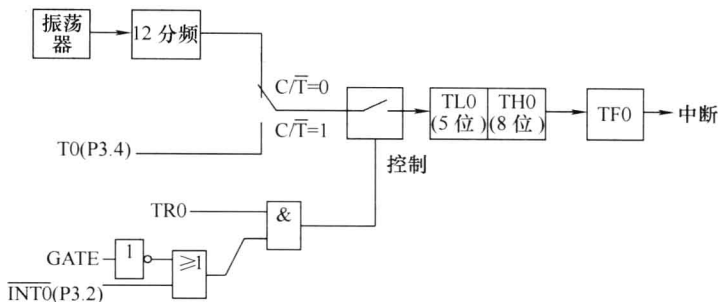


图 6-14 定时/计数器 T0 方式 0 逻辑结构

在图 6-14 中, 有两个电子开关, 其中一个为定时/计数器 T0 定时还是计数方式的选择开关, 一个是定时/计数器是否计数允许的控制开关。

计数器的计数脉冲由 C/ $\bar{T}$ 选择, 当 C/ $\bar{T}$ = 0 时, 结构图中电子开关打在上面, 以振荡器的输出信号经过 12 分频后信号作为 T0 的计数脉冲信号; 当 C/ $\bar{T}$ = 1 时, 结构图中电子开关打在下面, 计数器对 T0 引脚 (P3.4 引脚) 上输入的脉冲信号计数。

从结构图中可知, 当 GATE = 0 时, GATE 信号经过非门输出高电平, 经过或门输出总为高电平, 则控制信号完全由 TR0 决定。当 TR0 = 1 时, 则与门输出为 1, 控制开关控制端为高电平, 即开关闭合, 允许定时/计数器 T0 开始计数; TR0 = 0 时, 则与门输出为 0, 控制开关控制端为低电平, 即开关断开, 定时/计数器 T0 停止计数。

当 GATE = 1 时, 只有 P3.2 引脚输入高电平, 才能使得或门输出为高电平, 同时, TR0 必须为 1, 与门才能输出高电平, 开关闭合, 定时/计数器 T0 开始计数。当 $\overline{\text{INT0}}$ (P3.2) 引脚输入低电平时, 或门输出低电平, 无论 TR0 为 0 还是为 1, 与门输出总为低电平, 定时/计数器 T0 停止计数, 若 TR0 = 0, 无论 P3.2 引脚输入高电平还是低电平, 与门输出总为低电平, 定时/计数器 T0 停止计数。因此, 当 GATE = 1 时, 要让定时/计数器 T0 开始计数, 必须同时满足 TR0 = 1 和 $\overline{\text{INT0}}$ (P3.2) 引脚输入高电平两个条件。

定时/计数器 T0 工作于方式 0, 计数器开始工作时, 13 位计数器从计数初值开始加 1 计数, 当 13 位计数器各位全 1 后, 再加 1 计数, 则计数器计数溢出, 计数器计数溢出时, 由单片机硬件自动置位 TF0, 同时将 13 位计数器清 0。所以, 方式 0 下, 计数器的计数范围为 1 ~ 8192 (2^{13})。

T0 工作于方式 0 计数方式时, 计数初值为 a , 计数 N 次, 则计数器中的值为 $X = a + N$ 。

T0 工作于方式 0 定时方式时, 计数初值为 a , T0 从计数初值开始加 1 计数至计数溢出时, 时间为:

$$t = \frac{1}{f_{\text{osc}}} 12 (2^{13} - a) s$$

式中, f_{osc} 为系统晶振频率。

若单片机系统的晶振频率 $f_{osc} = 12\text{MHz}$, 则定时时间 $t = (2^{13} - a) \mu\text{s}$ 。

(2) 方式 1

当 T0 方式字段中 $M1M0 = 01$ 时, 定时/计数器工作方式 1, 方式 1 为 16 位的定时计数方式。在这种方式下, T0 为 16 位的定时/计数器; 方式 0 和方式 1 的差别仅仅在于计数器的位数不同, 其余的逻辑结构图完全相同。定时/计数器 T0 工作于方式 1 的逻辑结构框图如图 6-15 所示。16 位计数器, 由 TL0 的中的 8 位和 TH0 中的 8 位组成, TH0 中 8 位作为计数器的高 8 位, TL0 中的 8 位作为计数器的低 5 位, 当 TL0 计数溢出时向 TH0 进位, TH0 计数溢出时置“1”溢出标志 TF0。

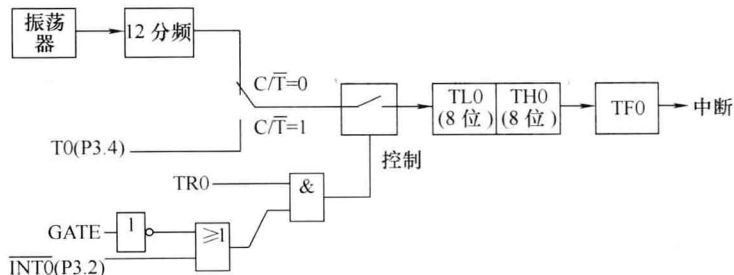


图 6-15 定时/计数器 T0 方式 1 逻辑结构

当 $GATE = 0$ 时, 只要 $TR0 = 1$, 则定时/计数器 T0 开始计数。

当 $GATE = 1$ 时, $TR0 = 1$, 同时 $INT0$ (P3.2) 输入高电平, 则定时/计数器 T0 开始计数。

计数器开始工作时, 16 位的计数器从计数初值 a 加 1 计数, 当 16 位计数器各位全 1 以后, 再加 1 次, 计数器产生溢出, 由单片机硬件系统自动置“1”TF0, 同时计数器清 0。方式 1 下, 计数器的计数范围为 $1 \sim 65536$ (2^{16})。

T0 工作于方式 1 定时方式时, 计数初值为 a , T0 从计数初值开始加 1 计数至计数溢出时, 时间为:

$$t = \frac{1}{f_{osc}} 12 (2^{16} - a) \text{ s}$$

式中, f_{osc} 为系统晶振频率。

若单片机系统的晶振频率 $f_{osc} = 12\text{MHz}$, 则定时时间 $t = (2^{16} - a) \mu\text{s}$ 。当计数初值 a 为 0 时, 定时时间最长, 最大定时时间为 $65536 \mu\text{s}$ 。

【例 6-5】 设 $f_{osc} = 12\text{MHz}$, 当定时/计数器 T0 工作于方式 1, 产生 50ms 定时, 试分别采用中断和查询的方式编程实现 P1.0 引脚产生周期为 1s 的方波。

解: 要使得 P1.0 引脚产生周期为 1s 的方波, 则 P1.0 引脚高低电平持续的时间应精确为 500ms , 本例使用定时/计数器 T0 精确定时 50ms , 则 10 次 50ms 中断时定时时间为 500ms 。如采用中断方式, 在 T0 中断服务程序中设置一个计数单元, 计数初值为 10 每次 50ms 中断时, 计数单元数值减 1, 当计数单元数值减为 0 时, 恰好是 10 次中断, 即时间为 500ms , 500ms 到来时引脚 P1.0 取反。

若采用查询的方式, 在主程序中不断查询 TF0 的状态, 当 TF0 为 1 时, 表示 50ms 定时时间已到, 在主程序中判断是否 10 次 50ms 定时已到, 如是则 P1.0 引脚取反, 若否, 则继

续循环等待。用查询方式，每次 TF0 置 1 后，TF0 必须由软件清 0。

1) 计数初值。根据公式：

$$t = \frac{1}{f_{\text{osc}}} 12 (2^{16} - a) s$$

则当定时时间为 50ms 时，计数初值 a 应该为

$$a = 2^{16} - \frac{f_{\text{osc}}}{12} t$$

代入 $f_{\text{osc}} = 12\text{MHz}$ ， $t = 50\text{ms}$ ，计算得

$$a = 15536 = 3\text{CB0H} (\text{十六进制})$$

即计数器中 TL0 的初值为 B0H；TH0 的初值为 3C H。

2) TMOD 定时/计数器寄存器。定时/计数器 T0 工作于方式 1，故 M1M0 = 01；工作于定时工作方式，C/T = 0；GATE = 0，则 TMOD = 01H。

① 中断方式。设置定时/计数器工作于方式 1，定时方式，同时中断允许。

主程序：

```

ORG 0000H
LJMP MAIN
ORG 000BH
LJMP INTT0
MAIN: MOV SP, #60H
      MOV TH0, #3CH
      MOV TL0, #0B0H
      MOV TMOD, #1           ; M1M0 = 01 T0 工作于方式 0, GATE = 0, C/T = 0 定时方式
      MOV IE, #82H          ; 允许 T0 向 CPU 申请中断
      SETB TR0               ; 启动 T0 开始计数
      MOV 30H, #10           ; T0 中断次数计数单元初始化为 10
      SJMP $

```

T0 中断服务程序：

```

INTT0: MOV TH0, #3CH
      MOV TL0, #0B0H        ; 重新赋计数器计数初值
      DJNZ 30H, RETT0       ; 计数单元减 1 不为 0 直接中断返回
      MOV 30H, #10          ; 减 1 为 0, 说明 500ms 时间已到, 为了下次继续定时 500ms
                                ; 计数单元赋计数初值 10
      CPL P1.0              ; P1.0 取反
      RETT0: RETI

```

在中断方式时，CPU 响应溢出中断请求时，自动把溢出标志位清 0。因此，在中断处理程序中无须对溢出标志位清 0。

C51 程序：

```

#include <reg52.h>
sbit P1_0 = P1^0;
unsigned char a = 10;
main ( )

```

```

{
    TH0 = 0x3c;          //T0 初始化
    TL0 = 0xb0;
    TMOD = 0x01;
    IE = 0x82;
    TR0 = 1;
    while (1);
}

timer0 ( ) interrupt 1 using 1
{
    TH0 = 0x3c;          //重赋计数初值
    TL0 = 0xb0;
    a -- ;
    if (a == 0)
    {
        a = 10;
        P1_0 = ~ P1_0;
    }
}

```

② 查询方式。采用查询方式，此时 T0 仍然工作方式 1 的定时方式，但设置为不允许 T0 向 CPU 申请中断。

程序如下：

```

ORG 0000H
MOV SP,#60H
MOV TH0,#3CH
MOV TL0,#0B0H
MOV TMOD,#1
MOV IE,#00H      ;关闭中断允许
MOV 30H,#10
SETB TR0
WAIT: JNB TF0,$    ;TF0 = 0 循环等待,TF0 = 1 继续往下执行
      CLR TF0      ;TF0 清 0,以便下次定时 50ms 的查询
      MOV TH0,#3CH ;50ms 定时的计数初值再次赋给定时/计数器 T0
      MOV TL0,#0B0H
      DJNZ 30H,WAIT ;计数单元减 1 不为 0,继续等待
      MOV 30H,#10  ;计数单元减 1 为 0,则再次把计数初值赋给计数单元
      CPL P1.0     ;P1.0 取反
      LJMP WAIT    ;返回 WAIT,继续查询等待

```

C51 程序：

```

#include <reg52.h>
sbit P1_0 = P1^0;
unsigned char a;

```

```

main ( )
{
    TH0 = 0x3c;
    TL0 = 0xb0;
    TMOD = 0x01;
    IE = 0;
    TR0 = 1;
    while(1)
    {
        for( a = 10; a > 0; a -- )
        {
            while ( TF0 == 0);    // TF0 为 0 则循环等待
            TF0 = 0;
            TH0 = 0x3c;
            TL0 = 0xb0;
        }
        P1_0 = ~ P1_0;           //10 次 T0 中断,P1.0 取反
    }
}

```

(3) 方式 2

当 T0 方式字段中 M1M0 = 10 时, 定时/计数器 T0 工作方式 2, 方式 2 为 8 位自动重装初值定时/计数方式。在这种方式下, 定时/计数器为 8 位自动重装初值的定时计数器, 逻辑结构框图如图 6-16 所示。

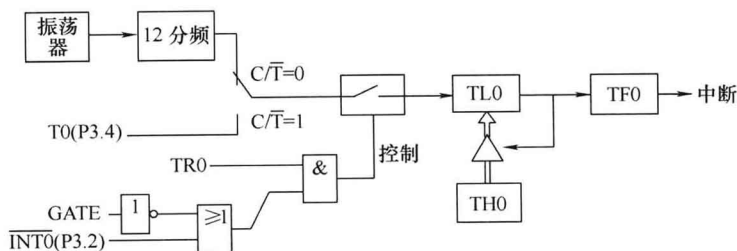


图 6-16 定时/计数器 T0 方式 2 逻辑结构

由 TL0 作为 8 位计数器, TH0 作为计数初值寄存器, 当 TL0 计数溢出时置“1”溢出标志 TF0, 同时将 TH0 中的内容自动送入 TL0 计数器中, TL0 计数器从这个计数初值开始计数, 使得 TL0 从这个计数初值开始加 1 计数, 计数溢出时再自动装入 TH0 中的计数初值, 周而复始。

当 T0 工作于方式 2 的计数和定时方式, 计数器中计数数值较小, 最大只能是 256 (2^8)。

T0 工作于方式 2 定时方式时, 计数初值为 a , T0 从计数初值开始加 1 计数至计数溢出时, 时间为:

$$t = \frac{1}{f_{\text{osc}}} 12(2^8 - a) \text{ s}$$

式中, f_{osc} 为系统晶振频率。

若单片机系统的晶振频率 $f_{\text{osc}} = 12\text{MHz}$, 则定时时间 $t = (2^8 - a) \mu\text{s}$ 。当计数初值 a 为 0 时, 定时时间最长, 最大定时时间为 $256\mu\text{s}$ 。

(4) 方式 3

当 T0 方式字段中 $M1M0 = 11$ 时, 定时/计数器 T0 工作于方式 3。只有定时/计数器 T0 有工作方式 3, 定时/计数器 T1 没有工作方式 3, 如果把 T1 设置为工作方式 3, 则 T1 停止计数。定时/计数器 T0 工作于方式 3 时逻辑结构如图 6-17 所示。此时, T0 分为两个独立的 8 位计数器 TL0 和 TH0, TL0 计数器使用定时/计数器 T0 的所有控制位和状态标志位: GATE、INT0 引脚 (P3.2)、T0 (P3.4)、TR0、TF0, TL0 作为一个独立的 8 位定时器或者外部事件计数器, TL0 计数溢出时置“1”溢出标志 TF0。同时, TL0 计数器清 0, TL0 的计数初值必须由软件赋值。

TH0 只能工作于 8 位的定时方式, 并借用定时/计数器 T1 的控制位和状态标志位 TR1 和 TF1。TR1 = 1 时, 定时/计数器 TH0 开始计数, 当 TH0 计数溢出时置“1”溢出标志 TF1。

在这种工作模式下, 单片机多出一个定时/计数器, 只有在 T1 用于波特率发生器时, 不需要溢出中断的场合, T0 才需要选工作方式 3, 以增加一个计数器。这时 T1 的运行由工作方式控制位 $M1M0$ 来控制, $M1M0 = 11$ 时, T1 停止计数, $M1M0 = 00, 01, 10$ 时允许计数, 计数溢出时并不置“1”标志 TF1。

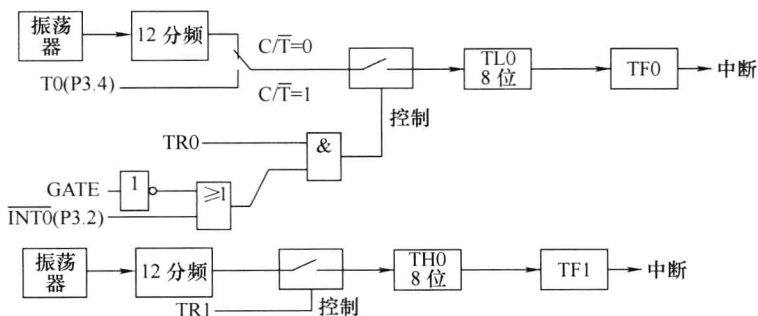


图 6-17 定时/计数器 T0 方式 3 逻辑结构

6.3.3 定时/计数器的初始化编程及应用

定时/计数器 T0、T1 可以用来实现精确定时或者对外部脉冲事件计数。下面通过一些具体的应用实例说明定时/计数器的使用方法。

【例 6-6】 占空比不同信号发生器。

设计一个应用程序, 使用定时/计数器 T0 定时方法在 P1.0 引脚输出周期为 100ms , 占空比为 1:9 的周期信号 (如图 6-18 所示), 晶振选用 12MHz 的晶振。

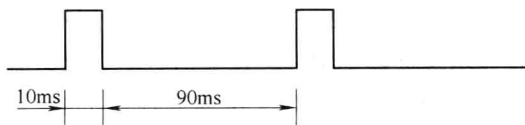


图 6-18 P1.0 引脚输出信号

由分析可知,周期 100ms,占空比为 1:9 的周期信号,高电平持续时间为 10ms,低电平持续时间为 90ms,使用定时/计数器 T0 产生 10ms 定时,T0 工作于方式 1,则计数初值如下:

$$t = 1\mu s(2^{16} - a) = 10\text{ms}$$

式中, $a = 65536 - 10000 = 55536 = \text{D8F0H}$ (十六进制)。

具体程序如下:

```

ORG 0000H
LJMP MAIN
ORG 000BH
LJMP PINT0
ORG 0030H
MAIN:  MOV SP,#60H
      MOV 31H,#9           ;设置一个计数器
      MOV TMOD,#01        ;定时/计数器初始化
      MOV TH0,#0D8H
      MOV TLO,#0 F0H
      MOV IE,#81H
      CLR P1.0
      CLR 00H             ;设置 00H 为位标志,为 1 时输出低电平,为 0 时输出高电平
      SETB TR0
HERE:  AJMP HERE
PINT0:  PUSH ACC
      PUSH PSW
      MOV TH0,#0D8H
      MOV TLO,#0 F0H
      JB 00H,LOW          ;若 00H 为 1,则输出 90ms
      SETB P1.0
      SETB 00H
      SJMP OUT
LOW:   CLR P1.0
      DJNZ 31H,OUT
      MOV 31H,#9
      CLR 00H
OUT:   POP PSW
      POP ACC
      RETI

```

【例 6-7】 秒表的程序设计。

单片机使用 12MHz 的晶振,定时/计数器 T0 工作于方式 2,产生 250μs 定时,每 1s 使秒表显示缓冲器 30H~32H 实时计时,缓冲器分配如下:30H 高 4 位为小时的十位,低 4 位为小时的个位;31H 高 4 位为分钟的十位,低 4 位为分钟的个位;32H 高 4 位为秒的十位,低 4 位为秒的个位。每 1 秒钟加 1 计时,当计时到 59 秒,下 1 秒到来时为 1 分钟,秒清 0,分钟加 1,当计时到 59 分 59 秒,下 1 秒到来时为 1 小时,分钟、秒清 0,小时加 1,能够计

时的最长计时时间为 99 小时 59 分 59 秒。

解：(1) 确定 TMOD

若定时/计数器 T0 工作于方式 2 的定时方式，则 TMOD 中各位如图 6-19。

D7	D6	D5	D4	D3	D2	D1	D0
GATE	C/ $\overline{T}$	M1	M0	GATE	C/ $\overline{T}$	M1	M0
*	*	*	*	0	0	1	0

图 6-19 TMOD 中各位

“*”表示任意，本例中没有用到定时/计数器 T1，所以 T1 的控制字段任意，T0 控制字段为 0010。

TMOD = 02H。

(2) 计算计数初值

由定时时间的计算公式

$$t = \frac{1}{f_{\text{osc}}} 12(2^8 - a) \text{ s}$$

当定时时间 $t = 250 \mu\text{s}$ ，晶振 $f_{\text{osc}} = 12\text{MHz}$ 时，计算计数初值 $a = 6$

电子钟每 1 秒，时、分、秒单元的值要加 1，T0 中断 4000 次时时间为 1 秒。

(3) 程序

```
ORG 0000H
LJMP MAIN           ;设置跳转到主程序
ORG 000BH           ;T0 中断服务程序入口地址
LJMP PINTT0         ;设置跳转到中断服务程序
ORG 0030H
MAIN: MOV SP,#60H   ;主程序
      MOV 36H,#10H
      MOV 37H,#0A0H ;4000 = (0FA0H) 中断次数放入计数单元,由于每次中断时,
                        ;中断次数减 1,两次使用 DJNZ 指令,故高位字节修正为 10H
      MOV TMOD,#02H
      MOV TL0,#6
      MOV TH0,#6
      MOV IE,#82H
      SETB TR0
      SJMP $
PINTT0: PUSH PSW      ;T0 中断服务程序
        PUSH ACC
        SETB RS0      ;选择 1 区的工作寄存器
        DJNZ 37H,RETURN
        DJNZ 36H, RETURN ;4000 次中断未到中断返回
        MOV 36H,#10H
        MOV 37H,#0A0H ;4000 次中断到来,计数器单元恢复初始值
        MOV R0,#32H
        MOV A,@R0
```

```

    ADD A,#1
    DA A                                ;调整为十进制相加
    MOV @R0,A
    CJNE A,#60H,RETURN                 ;修改秒表计时值,秒单元加1,不为60秒中断返回
    MOV @R0,#0                         ;为60秒,秒单元清0,分钟单元加1
    DEC R0
    MOV A,@R0
    ADD A,#1
    DA A
    MOV @R0,A
    CJNE A,#60H,RETURN                 ;若为60分钟,分钟单元清0,小时单元加1
    MOV @R0,#0
    DEC R0
    MOV A,@R0
    ADD A,#1
    DA A
    MOV @R0,A
RETURN: POP ACC                        ;恢复现场
    POP PSW
    RETI                               ;中断返回

```

C51 程序:

```

#include <reg52.h>
unsigned int a=4000;
unsigned char time_buf[] = {0,0,0};
main ( )
{
    TH0 = 0x06;
    TL0 = 0x06;
    TMOD = 0x02;
    IE = 0x82;
    TR0 = 1;
    while(1) { }
}

timer0 ( ) interrupt 1 using 1        //T0 的中断函数
{
    a -- ;
    if (a == 0)
    {
        a = 4000;
        time_buf[2] + = 1;            //秒单元加1
        if(time_buf[2] > = 60)        //若为60秒
        {
            time_buf[2] = 0;          //秒单元清0

```

```

time_buf[1] + = 1;           //分钟单元加 1
if (time_buf[1] > = 60)      //若为 60 分
{
    time_buf[1] = 0;         //分钟单元清 0
    time_buf[0] + = 1;       //小时单元加 1
}
}
}
}

```

6.3.4 AT89S 系列单片机看门狗定时器的编程方法

目前,大多数单片机内都有看门狗监视定时器(Watchdog Timer)。单片机应用系统工作过程中一旦出现程序执行跑飞的现象,如硬件正常工作而程序因受到干扰而出现了不正确跳转,转入一个非法区域执行程序,或者出现异常死循环。此时,通过看门狗监视定时器监视这一现象。如发现程序跑偏,则看门狗给单片机一个复位信号,使得单片机系统复位,复位后单片机内部寄存器恢复成初始状态,软件程序从 0000H 地址开始重新执行程序。

看门狗监视定时器是一个 n 位的计数器,开启看门狗定时器后,由计数器对单片机内部的机器周期进行计数,计数器的计数初值为 0,当计数器计数溢出时,监视器输出高电平复位信号,使单片机系统复位。计数器的溢出时间为 $2^n T_{cy}$,如果程序正常执行,不需要对单片机系统复位,程序中每隔一段时间(小于溢出时间),对看门狗监视定时器复位(喂狗)一次,监视定时器复位恢复到计数初值为 0 的状态,继续计数,在计数溢出之前,对其进行复位,这样保证程序正常执行时,看门狗不会溢出,也就不输出高电平复位信号。

例如,AT89S52 看门狗定时器由一个 13 位定时器及 WDTRST(字节地址为 6AH)寄存器构成。开启看门狗定时器后,13 位定时器会自动加 1 计数,如不对定时器复位,则每计数 8192 (2^{13}) 个机器周期溢出一次,并产生一个高电平复位信号,使单片机系统复位。对于 12MHz 的时钟脉冲每 8192 μ s(约 8.192ms)产生一个复位信号,启动看门狗定时器,当系统超过 8.192ms 没有对看门狗定时器复位,看门狗定时器溢出,让系统复位。为了系统既能正常工作,又不会出现死机(程序跑飞),大约在 8ms 内必须对看门狗定时器复位(喂狗)一次。

启动看门狗的命令格式如下:

```

MOV    0A6H, #1EH
MOV    0A6H, #0E1H    ;启动看门狗

```

对 0A6H 单元依次写入数据 1EH 和 0E1H,激活看门狗监视定时器,如果程序正常运行,在看门狗监视定时器溢出时间前,再次复位看门狗监视定时器。复位的方法和激活看门狗监视定时器方法一样,仍然是将数据 1EH 和 0E1H 依次写入 0A6H 单元。

如果用 C51 语言来完成此功能,程序如下:

```

#include <reg52.h>
sfr    WDTRST = 0xa6;
main()
{
    .....
    WDTRST = 0x1e;           //启动看门狗
}

```



```

WDTRST = 0x01;
While(1)
{
    WDTRST = 0x01;           //喂狗
    WDTRST = 0x01;
    .....
}

```

这部分执行时间必须少于 8ms(对 12MHz 时钟)

6.4 AT89 系列单片机的串行接口及串行通信

6.4.1 串行口的基本通信方式

单片机 CPU 和外部设备进行数据交换称为通信，常用的通信方式有两种：并行通信和串行通信。

并行通信是指数据的各位同时并行传送的通信方式。多位数据同时通过多根数据线传送，每一根数据线传送一位二进制代码。并行通信的优点是传输速度快，其缺点是占用的硬件资源多，适用于近距离传输和处理速度较快的场合，多用于计算机内部。

串行通信是指数据按时间先后顺序一位一位串行地顺序传送数据的通信方式。与并行通信相比，串行通信占用较少的通信线，硬件成本降低，适合于较远距离的数据传输，如计算机与计算机之间、计算机与外部设备之间远距离通信。其缺点是传输速度慢、效率低。

单片机的串行通信通过串行口来实现，51 系列单片机提供一个全双工的异步串行接口，用于串行口接收和发送数据。常用的串行通信有 3 种方式：单工方式、半双工方式和全双工方式，如图 6-20 所示。

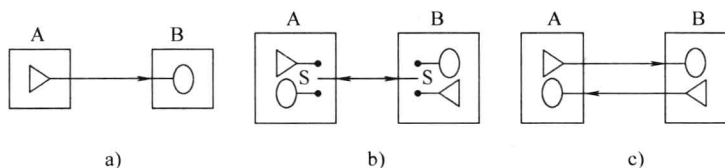


图 6-20 单片机的通信方式

a) 单工 b) 半双工 c) 全双工

单工方式下，数据只能单方向传输，只能从一端向另一端传输，不能往相反的方向传输；半双工方式下，允许数据向两个相反的方向传输，但在同一时刻数据只能向一个方向传输；全双工方式下，数据可以同时向两个相反的方向传输。需要两根独立的数据线，一根数据线用来发送数据，一根数据线用来接收数据。

在串行通信中，有两种最基本的通信方式：异步通信方式和同步通信方式。

异步通信方式是按字符传送的，用一个起始位表示字符的开始，接着是数据位和校验位，最后用一位停止位表示字符的结束，这样构成一帧的数据。在异步通信中，每个数据都以这样的帧形式传送，数据在通信线上一位一位地串行传送。异步通信方式一帧的数据格式如图 6-21 所示，起始位一般占用 1 位，用低电平表示，接着传输 1 字节的数据、校验位和

停止位。

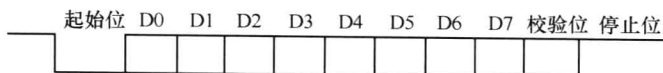


图 6-21 异步通信数据格式

在异步通信时，通信双方的字符格式必须是相同的，同时双方通信的波特率也必须相同。波特率是指每秒钟内传输的二进制数的位数。例如，串行通信的波特率为 2400bit/s，表示每秒钟内传输 2400 位二进制数。若字符格式为：1 位数据位，8 位数据位，无校验位，1 位停止位，这样每秒钟内数据传输为 240 个字符。

在异步通信方式中，每一个字符的发送都需要加上起始位和停止位，占用了时间，所以，在数据块传输时，为了提高速度，就去掉这些标志，采用同步传送。同步通信方式按数据块传输，传输的多个字符，按先后顺序组成数据块，在数据块前面加上同步字符，作为数据块的起始符号，在数据块的后面加上校验字符，用于校验数据传输的错误。同步通信的数据块格式如图 6-22 所示，在同步通信中字符之间没有间隔，通信效率较高。

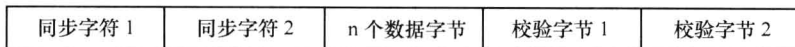


图 6-22 同步通信数据格式

6.4.2 单片机串行口及控制寄存器

AT89 系列单片机提供一个可编程全双工的串行通信接口，可以同时发送和接收数据。串行口内部有发送缓冲器和接收缓冲器，在物理构造上是两个独立的缓冲器，但是，它们共享一个字节地址 98H，共享一个特殊功能寄存器 SBUF，把数据写入 SBUF，则自动把数据送入发送缓冲器，从 SBUF 中读取数据，则自动读取接收缓冲器中的数据。

AT89 单片机串行口只有两个控制寄存器：串行口控制寄存器 SCON，用来选择串行口的工作方式，控制数据的接收和发送，给出串行口的当前工作状态等。还用到特殊功能寄存器 PCON 其中的一位，用来控制串行口的波特率。

1. 串行口控制寄存器 SCON

串行口控制寄存器 SCON，字节地址为 98H，是一个用于串行口控制的特殊功能寄存器，共 8 位，每一位具有位寻址功能。SCON 的数据位格式如图 6-23 所示。

D7	D6	D5	D4	D3	D2	D1	D0
SM0	SM1	SM2	REN	TB8	RB8	TI	RI

图 6-23 串行口控制寄存器 SCON

SM0、SM1 是串行口工作方式选择位，共有 4 种工作方式，其功能如表 6-3 所示。

表 6-3 串行口工作方式及其功能

SM0	SM1	工作方式	功能说明	波特率
0	0	方式 0	移位寄存器方式	$f_{osc}/12$
0	1	方式 1	8 位异步通信方式	可变
1	0	方式 2	9 位异步通信方式	$f_{osc}/64$, $f_{osc}/32$
1	1	方式 3	9 位异步通信方式	可变

SM2 是方式 2 和方式 3 的多机通信控制位。对于方式 2 和方式 3，如 SM2 = 1，接收到的第 9 位数据（RB8）为 0 时不置位 RI，则接收到的数据丢失，只有接收到的第 9 位数据（RB8）为 1 时，才将接收到的前 8 位数据送入 SBUF，并置位 RI，产生中断请求。当 SM2 = 0 时，不论第 9 位接收到的是 0 还是 1，都将接收到前 8 位数据送入 SBUF 中，并将 RI 置 1，产生中断请求。

对于方式 1，如果 SM2 = 1，则只有接收到有效的停止位时才会置位 RI。对于方式 0，SM2 应该为 0。

REN 是串行接受允许控制位。REN = 1 时，允许串行口接受数据；REN = 0 时，禁止串行口接收数据。

TB8 是方式 2 和方式 3 时发送数据的第 9 位数据，TB8 必须由软件清 0 或者置 1，TB8 中的数据是方式 2 和方式 3 需要发送的第 9 位数据。

RB8 是方式 2 和方式 3 时接收到的第 9 位数据。在方式 1 中，如果 SM2 = 0，RB8 是接收到的停止位，在方式 0 中，不使用 RB8。

TI 是串行口发送中断标志位：当串行口工作于方式 0 时，发送完第 8 位数据，硬件自动将 TI 置 1。当串行口工作于其他 3 种工作方式，串行口开始发送停止位时，硬件自动将 TI 置 1。当 TI = 1 时，表示一帧数据发送完毕，并向 CPU 请求中断。TI 必须由软件清 0。

RI 是串行口接收中断标志位：当串行口工作于方式 0 时，接收完第 8 位数据，硬件自动将 RI 置 1。当串行口工作于其他 3 种工作方式，串行口接收到停止位时，硬件自动将 RI 置 1。当 RI = 1 时，表示一帧数据接收完毕，并向 CPU 请求中断。RI 必须由软件清 0。

2. 电源控制寄存器 PCON

电源控制寄存器 PCON 的字节地址为 87H，数据格式如图 6-24 所示。

D7	D6	D5	D4	D3	D2	D1	D0
SMOD	—	—	—	GF1	GF0	PD	IDL

图 6-24 电源控制寄存器 PCON

PCON 中只有一位 SMOD 与串行口工作有关系。

SMOD 波特率加倍选择位：串行口工作在方式 1、方式 2、方式 3 时，如果 SMOD = 1，则串行口波特率加倍；若 SMOD = 0，则波特率不会提高。

6.4.3 单片机串行通信工作方式

AT89 系列单片机的串行接口有 4 种工作方式，它们由 SCON 中 SM0 和 SM1 这两位定义。下面分别说明串行口 4 种工作方式的功能特性和工作原理。

1. 方式 0

串行口工作方式 0 为同步移位寄存器工作方式，主要用于扩展并行输入/输出口。方式 0 输出时将发送缓冲器 SBUF 中的内容串行地移到外部寄存器，输入时将外部移位寄存器内容移入内部移位寄存器，然后再写入接收缓冲器 SBUF。

以方式 0 工作时，数据由 RXD 引脚输入或者输出，同步移位脉冲由 TXD 引脚输出，波特率为单片机振荡频率的 1/12。TXD 引脚输出的同步移位脉冲，每一个脉冲周期内完成一位二进制数据的接收或者发送。

(1) 方式 0 发送

方式 0 发送时, 数据由 RXD 引脚串行输出, TXD 引脚输出同步移位脉冲信号, CPU 对发送数据缓冲器 SBUF 写入数据, 例如 “MOV SBUF, A”, 产生一个正脉冲, 启动串行口发送器, 将数据从 RXD 引脚串行输出, 串行输出的波特率固定, 系统振荡频率的 1/12, 当输出完第 8 位数据后, 单片机自动置位 TI 标志位。

方式 0 输出时, 串行口上可以外接串行输入/并行输出的移位寄存器, 如 74LS164, 用以扩展并行输出口, 如图 6-25 所示。串行数据由 RXD 端输出, TXD 端输出的同步移位脉冲将 RXD 端输出的数据逐位移入 74LS164, 低位 D0 先移出, 当 8 位全部移完时, 数据 D0 ~ D7 分别出现在 74LS164 相应的引脚, 同时, 把 TI 置位, 表示 1 字节的数据发送完毕。若要发送下一字节的数据, 必须先用软件将 TI 清 0。然后再重复上述过程。

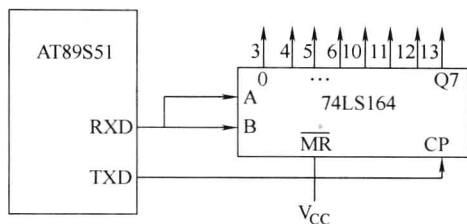


图 6-25 方式 0 扩展并行输出口

(2) 方式 0 接收

串行口工作于方式 0 接收数据时, 可以扩展一片或者多片并行输入/串行输出的移位寄存器, 如 74LS165, 如图 6-26 所示。并行输入的数据从 74LS165 的并行数据输入引脚输入, 由 74LS165 转换为串行数据, 输入单片机的 RXD 引脚, 单片机 TXD 引脚输出移位脉冲信号。在 REN = 1, RI = 0 时启动串行口接收, TXD 引脚输出的移位脉冲将并行数据逐一移入 RXD 端。8 位数据全部移入, 则置位 RI, 单片机执行指令 MOV A, SBUF, 取走接收缓冲器中的数据, 如果需要再接收, 则必须先用软件将 RI 清 0。

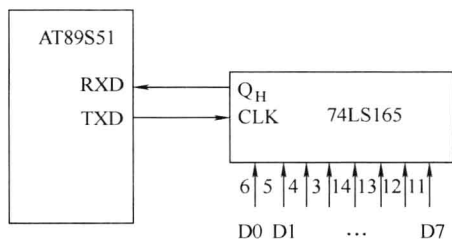


图 6-26 方式 0 扩展并行输入口

2. 方式 1

串行口工作方式 1 是 8 位的异步通信方式, TXD 为串行数据输出线, RXD 为串行数据输入线, 传送一帧信息的数据格式为: 1 位低电平起始位, 8 位数据位 (先低位后高位), 1 位高电平停止位。方式 1 发送和接收数据的波特率可变, 由定时/计数器 T1 或者 T2 的溢出率决定, 是可变的。

(1) 方式 1 发送

当 TI = 0 时, 执行指令 “MOV SBUF, A”, CPU 向串行口发送缓冲器 SBUF 写入 1 字节的数据, 启动串行口发送, 在串行口内部产生的移位脉冲作用下 (控制波特率), 在 TXD 引脚上输出一帧信息, 先发送起始位 0, 接着从低位开始输出 8 位数据, 最后输出停止位 1, 发送后置位中断标志 TI, 输出完一个字符后串行口停止工作, CPU 执行程序判断 TI 为 1 后软件清 “0” TI 中断标志, 再向 SBUF 写入数据, 启动串行口发送下一字符。

(2) 方式 1 接收

在 REN = 1 和 RI = 0 时, 允许串行口接收器接收数据, 接收器以所选波特率的 16 倍速采样 RXD 端电平, 检测到 RXD 端输入电平发生负跳变时, 复位内部的 16 分频计数器, 计

数器的 16 个状态把传送一位数据的过程等分成 16 个状态，在 7, 8, 9 这 3 个状态检测 RXD 引脚的输入电平，3 次采样的值中至少有两次相同的值，这样可以防止外界的干扰。如果在第一位接收数据时间内接收到的值不为 0，说明它不是一帧数据的起始位，起始位无效，复位接收电路，重新检测 RXD 端的负跳变，如果起始位有效，则开始接收本帧数据的其余部分信息，接收到停止位为 1 时，将接收到的 8 位数据装入接收数据缓冲器 SBUF，置位 RI，表示串行口接收到有效的一帧信息，向 CPU 请求中断。CPU 响应中断时，取走 SBUF 接收数据缓冲器中已经接收的 1 字节数据，如“MOV A, SBUF”，并清“0”RI 接收中断标志位，如 CLR RI。接着串行口输入控制电路重新搜索 RXD 端上电平负跳变，接收下一个数据。

3. 方式 2 和方式 3

串行口工作于方式 2 和方式 3 时，是工作于 9 位的异步通信方式，TXD 为数据发送端，RXD 为数据接收端，传送一帧信息的数据格式为：1 位低电平起始位，8 位数据位（先低位后高位），1 位附加的第 9 位数据位，1 位高电平停止位，数据格式如图 6-27 所示。方式 2 发送和接收数据的波特率固定为振荡频率的 1/64 或者 1/32，而方式 3 的波特率由定时/计数器 T1 或者 T2 的溢出率决定。

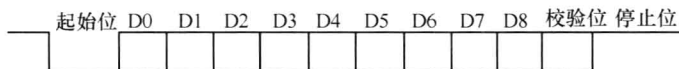


图 6-27 串行口方式 2（方式 3）数据格式

（1）方式 2 和方式 3 发送

方式 2 和方式 3 发送数据的过程类似于方式 1，但不同的是，方式 2（方式 3）多了第 9 位附加数据。发送数据时，除了把 1 字节数据写入 SBUF 发送缓冲器外，还需要把第 9 位数据写入 SCON 的 TB8。

具体工作过程为：当 TI = 0 时，CPU 预先要把要发送的第 9 位数据写入 SCON 的 TB8 位，接着 CPU 向 SBUF 发送缓冲器中写入 1 字节的待发送数据，启动串行口发送，串行口在内部移位脉冲控制下，先发送起始位 0，接着从低位开始依次发送 SBUF 中的 8 位数据，再发送第 9 位数据 TB8，最后发送停止位，发送完数据自动置位 TI 发送中断标志位，CPU 判断 TI 为 1 后，软件清“0”TI 位后，可以发送下一帧数据。

（2）方式 2 和方式 3 接收

方式 2 和方式 3 的接收过程也与方式 1 类似。不同的是，方式 1 中 RB8 中存放的是停止位，方式 2（方式 3）中 RB8 中存放的是第 9 位数据。

在 REN = 1 和 RI = 0 时，允许串行口接收器接收数据，接收器以所选波特率的 16 倍速采样 RXD 端电平，检测到 RXD 端输入电平发生负跳变时，复位内部的 16 分频计数器，计数器的 16 个状态把传送一位数据的过程等分成 16 个状态，在 7, 8, 9 这 3 个状态检测 RXD 引脚的输入电平，3 次采样的值中至少有两次相同的值，这样可以防止外界的干扰。如果在第一位接收数据时间内接收到的值不为 0，说明它不是一帧数据的起始位，起始位无效，复位接收电路，重新检测 RXD 端的负跳变，如果起始位有效，则开始接收本帧数据的其余部分信息。

接收到停止位为 1 时, 若 $SM2 = 0$, 串行口工作于单机通信方式, 则将接收到的 8 位数据装入接收数据缓冲器 SBUF, 第 9 位数据装入 RB8, 置位 RI, 表示串行口接收到有效的一帧信息, 向 CPU 请求中断。若 $SM2 = 1$, 串行口工作于多机通信方式, 当接收到第 9 位数据 $RB8 = 1$ 时, 数据有效接收, 将数据分别装入 SBUF 和 RB8 中, 置位 RI, 当接收到第 9 位数据 $RB8 = 0$ 时, 数据丢失, 并且不置位 RI。

若需要串行口继续接收下一帧数据, 则软件清“0”RI 位, $REN = 1$, 一位时间后又开始搜索 RXD 上的负跳变。

6.4.4 单片机串行口的初始化编程及波特率设置

1. 串行口波特率

波特率是串行口数据通信的一个重要指标, 表示串行口发送和接收数据的速度, 波特率对数据的传送成功至关重要, 为保证通信的可靠性, 发送和接收方波特率相对误差不大于 2.5%。串行口的几种工作方式对于波特率的选择有所不同, 具体如下。

(1) 方式 0 的波特率

串行口方式 0 的波特率是固定的, 由振荡频率确定, 是振荡频率的 1/12, 即

$$\text{方式 0 波特率} = \text{振荡频率} / 12$$

例如, 单片机系统选择 12MHz 的振荡频率, 则方式 0 发送和接收数据的波特率为 1MHz。

(2) 方式 2 的波特率

串行口方式 2 的波特率也是固定的, 为振荡频率的 1/64 或者 1/32, 这由 PCON 控制寄存器中的 SMOD 位决定。当 $SMOD = 0$ 时, 串行口方式 2 的波特率是振荡频率的 1/64; 当 $SMOD = 1$ 时, 串行口方式 2 的波特率是振荡频率的 1/32, 可以表示为

$$\text{方式 2 波特率} = 2^{SMOD} \times \text{振荡频率} / 64$$

(3) 方式 1 和方式 3 的波特率

串行口工作于方式 1 和方式 3 时, 波特率由定时/计数器 T1 或 T2 的溢出率和 SMOD 位一起确定。由于 T1 和 T2 可以编程确定其计数初值, 则溢出时间可选择范围比较大, 因此溢出率可选择范围大, 故方式 1 和方式 3 串行通信的波特率是可变的, 也是串行通信中最常用的工作方式。

1) T1 为波特率发生器。单片机复位后, 若串行口工作于方式 1 或者方式 3, 则默认为定时/计数器 T1 为波特率发生器。此时, 串行口的波特率为

$$\text{方式 1 (方式 3) 波特率} = 2^{SMOD} \times (\text{T1 溢出率}) / 32$$

$SMOD = 0$ 时, 波特率等于 T1 溢出率的 1/32; $SMOD = 1$ 时, 波特率加倍, 等于 T1 溢出率的 1/16。

T1 的溢出率是定时/计数器 T1 溢出时间的倒数分之一, 当定时器 T1 作为波特率发生器时, 禁止 T1 中断, T1 工作于定时方式, 一般选择工作方式 2 (8 位自动重装初值工作方式), 若计数初值为 a 。此时, T1 的溢出时间为

$$t = 12 \frac{1}{f_{osc}} (2^8 - a)$$

式中, f_{osc} 为单片机系统的振荡频率。

T1 的溢出率为溢出时间的倒数。此时，波特率的计算公式为

$$\text{方式 1 (方式 3) 波特率} = 2^{\text{SMOD}} f_{\text{osc}} / (32 \times 12 \times (2^8 - a))$$

表 6-4 列出了常用的波特率对应的振荡频率、T1 的计数初值以及波特率的相对误差，T1 工作于方式 2 的定时工作方式。

表 6-4 常用波特率与 T1 参数

波特率/ (bit/s)	振荡频率/MHz	SMOD	T1 计数初值	实际波特率 / (bit/s)	误差 / (bit/s) (%)
2400	12	0	F3H	2404	0.16
1200	12	0	E6H	1202	0.16
19200	11.0592	1	FDH	19200	0
9600	11.0592	0	FDH	9600	0
4800	11.0592	0	FAH	4800	0
2400	11.0592	0	F4H	2400	0
1200	11.0592	0	E8H	1200	0

当振荡频率选用 11.0592MHz 时，对于常用的标准波特率，根据选定的计数初值，可以达到波特率的相对误差为 0，所以这个频率是最常用的频率。

2) T2 为波特率发生器。在某些型号的单片机中有定时/计数器 T2 的，单片机复位后，TCLK = RCLK = 0，当设置串行口工作于波特率可变工作方式时，T1 为默认定时/计数器；若将 TCLK、RCLK 置 1，则以 T2 为串行口波特率发生器，T2 工作于波特率发生器方式，具体参考相关资料。

2. 串行口初始化编程

应用串行口发送或者接收数据，首先要对串行口初始化编程，选择串行口的工作方式、串行口的波特率以及是否允许串行口中断等，也就是对特殊功能寄存器 SCON、PCON、TMOD、TCON、TH1、TL1、IE、IP、SBUF 等编程。

串行口初始化编程一般考虑如下几个方面。

1) 确定波特率。如果串行口工作于波特率可变方式，确定 T1 为波特率发生器，还是 T2 为波特率发生器，确定定时/计数器的工作方式和计数初值，设定其为中断不允许，并完成其初始化编程。相关寄存器有 PCON、TMOD、TCON、TH1、TL1、IE。

2) 确定串行口的工作方式。设置是否接收允许，以及 TI、RI 标志位，如有第 9 位数据，设置 TB8，这些都通过对串行口控制寄存器 SCON 初始化编程确定。

3) 确定串行口是否中断允许及其中断优先级。通过对特殊功能寄存器 IE、IP 初始化编程确定。

【例 6-8】 已知振荡频率 $f_{\text{osc}} = 11.0592\text{MHz}$ ，对串行口初始化编程，设置其工作于方式 1 发送/接收方式，波特率为 9600bit/s，允许串行口中断。

解：1) 波特率 9600bit/s

方式 1 的波特率是可变的，当振荡频率 $f_{\text{osc}} = 11.0592\text{MHz}$ ，波特率为 9600bit/s，可以使用定时/计数器 T1 为波特率发生器；工作于方式 2（8 位自动重装初值工作方式）定时方式，计数初值为 FDH，不允许 T1 中断。

- 2) 串行口工作于方式 1, 8 位异步通信方式, 则 $SCON = 50H$ 。
3) 允许串行口中断, 则 $IE = 90H$ 。

初始化程序段如下:

```
MOV  TMOD,#20H      ;T1 工作于方式 2 定时方式
MOV  TH1,#0FDH
MOV  TL1,#0FDH      ;设置计数初值
SETB  TR1           ;启动 T1
MOV  SCON,#50H      ;串行口工作于方式 1,RI=0,TI=0
MOV  IE,#90H        ;设置串行口中断允许
```

6.4.5 RS-232C 串行口标准

微型计算机和单片机系统大都采用总线结构, 单片机常用的总线有并行总线和串行总线。常用的串行总线有 RS-232C、RS-422、RS-485 总线。

RS-232C 是美国电子工业协会 (Electronic Industries Association, EIA) 公布的串行总线标准, 用于实现数字设备之间的数据通信, 通信距离最大为 15m, 传输速率为 20kbit/s。

1. RS-232C 的电气特性及帧格式

RS-232C 的电气标准是: $-12 \sim -5V$ 为逻辑电平 1; $+5 \sim +12V$ 为逻辑电平 0。

RS-232C 具有如下主要电气特性。

- 1) 带 $3 \sim 7k\Omega$ 负载时驱动器的输出电平是: $-12 \sim -5V$ 为逻辑电平 1; $+5 \sim +12V$ 为逻辑电平 0。
- 2) 不带负载时驱动器的输出电平为 $-25 \sim +25V$ 。
- 3) 输出短路电流为 0.5A。
- 4) 驱动器转换速率小于 $30V/\mu s$ 。
- 5) 接收器输入阻抗为 $3 \sim 7k\Omega$ 。
- 6) 接收器输入电压的允许范围为 $-25 \sim +25V$ 。
- 7) 输入开路时接收器的输出为逻辑 1。
- 8) $+3V$ 输入时接收器的输出为逻辑 0。
- 9) $-3V$ 输入时接收器的输出为逻辑 1。
- 10) 最大负载电容为 2500pF。

RS-232C 一帧数据的格式为 1 位起始位, 8 位数据位, 1 位奇偶校验位, 1 位或 1.5 位或 2 位停止位。

常用的 RS-232C 连接器有 DB-25 和 DB-9。完整的 RS-232C 总线由 25 根信号线组成。DB-25 是 RS-232C 总线的标准连接器, 上面有 25 根引脚; DB-9 是简约型的 RS-232C 总线, 上面有 9 根引脚。具体信号线和引脚号对应如表 6-5 和表 6-6。

表 6-5 DB-25 引脚

引脚号	符号	名称	说明
1	PGND	保护地	
2	TXD	数据发送线	
3	RXD	数据接收线	

(续)

引脚号	符号	名称	说明
4	RTS	请求发送	
5	CTS	允许发送	
6	DSR	数据准备就绪	
7	SGND	信号地	
8	DCD	接收信号载波检测	收到一个满足一定标准的信号时置位
9		未定义	
10		未定义	
11		未定义	
12	RCD	辅助信道接收检测	
13	CTS	辅助信道允许发送	
14	TXD	辅助信道数据发送线	辅助信道传输较主信道低
15		发送信号元定时	
16	RXD	辅助信道数据接收线	
17		接收信号元定时	
18		未定义	
19	RTS	辅助信道请求发送	
20	DTR	数据终端准备就绪	
21	SD	信号检测	根据数据信息是否有错置位或复位
22	RI	振铃指示	收到振铃信号时置位
23		数据信号速率选择	指定两种传输速率中的一种
24		外部发送时钟	
25		未定义	

表 6-6 DB-9 引脚

引脚号	引脚符号	引脚名称
1	DCD	接收信号载波检测
2	RXD	数据接收线
3	TXD	数据发送线
4	DTR	数据终端准备就绪
5	GND	地线
6	DSR	数据准备就绪
7	RTS	请求发送
8	CTS	清除发送
9	RT	振铃指示

2. RS-232C 与 TTL 的电平转换

在单片机应用系统中，单片机常常作为 PC 的前置机，单片机完成数据采集，通过串行口把数据传输给 PC，PC 完成数据的存储、处理和分析等功能。

MCS-51 系列单片机的串行口电平为 TTL 电平标准, 大于 0.8V 为 1, 小于 0.3V 为 0, 而 PC 串行口的电平标准为 RS-232C 标准, 则当单片机串行口与 PC 串行口相连接时必须进行电平转换。

常用的 RS-232C 电平转换器芯片有 MAX232、HI232、1488、1489 等, 它们的功能大致相同。如图 6-28 所示的 MAX232 电平转换器, 只需要 +5V 供电, 芯片内部有 2 组 TTL 电平到 RS-232C 电平转换器, 有 2 组 RS-232C 电平到 TTL 电平转换器, 从 T1IN 引脚 (11 脚) 输入 TTL 电平, 从 T1OUT 引脚 (14 脚) 输出 RS-232C 电平; 从 T2IN 引脚 (10 脚) 输入 TTL 电平, 从 T2OUT 引脚 (7 脚) 输出 RS-232C 电平; 从 R1IN 引脚 (13 脚) 输入 RS-232C 电平, 从 R1OUT 引脚 (12 脚) 输出 TTL 电平; 从 R2IN 引脚 (8 脚) 输入 RS-232C 电平, 从 R2OUT 引脚 (9 脚) 输出 TTL 电平。

单片机和 PC 电平转换结构如图 6-29 所示。单片机的 TTL 电平经过电平转换器与 PC 的 RS-232C 电平相连接。单片机串口发送线 TXD, 经过电平转换, 转换为 RS232 电平, 接入 PC 串口接收线 RXD, PC 串口发送线 TXD, 经过电平转换, 转换为 TTL 电平, 接入单片机的串口接收线 RXD。

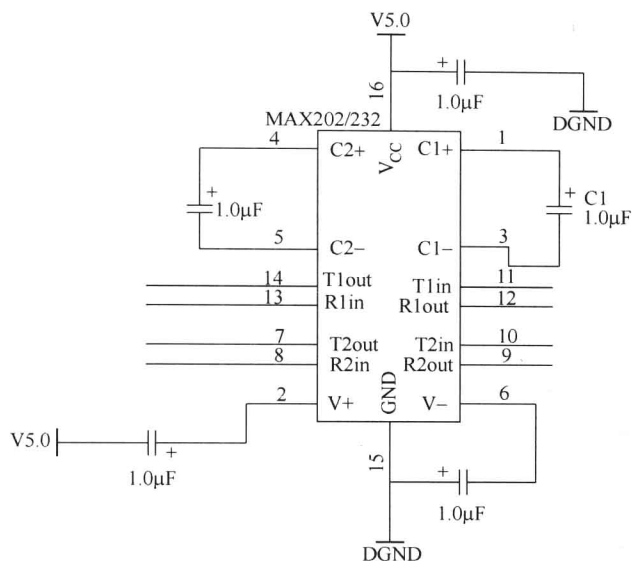


图 6-28 MAX232 电平转换器

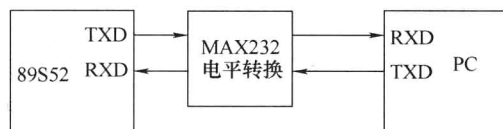


图 6-29 单片机和 PC 的通信

6.4.6 RS-422、RS-485 标准串行总线接口

为了解决 RS-232C 标准传输距离短的缺点, 随之 EIA 又推出了一些新的串行总线标准, 如 RS-422、RS-485 标准。

1. RS-422 标准接口

RS-422A 的最大传输距离是 300m, 传输速率为 10Mbit/s, 在低速方式下, 最大传输距离可达 1200m。

RS-422A 与 RS-232C 的主要区别是信号的传输方式不同。RS-232C 利用信号线与信号地之间的电压差传输信号; 在 RS-422A 的发送端把逻辑电平转换成电压差进行传输, 在接收端把电压差转换成逻辑电平, 并允许在一条平衡总线上最多连接 10 个接收器。RS-422A 是一种单机发送多机接收的单向、平衡传输规范。

TTL 电平到 RS-422A 电平转换器常用发送驱动器 SN75174, RS-422A 电平到 TTL 电平常用传输线接收器 SN75175, 引脚图如图 6-30 所示。

RS-422 接口电路如图 6-31 所示, 75174 完成 TTL 电平到 RS-422A 电平转换, 经过数据线传输, 75175 把接收到的 RS-422A 电平转换为 TTL 电平输出。

2. RS-485 标准接口

EIA 又于 1983 年在 RS-422 基础上制定了 RS-485 标准, 增加了多点、双向通信能力, 允许多个发送器连接到同一条总线上, 同时增加了发送器的驱动能力和冲突保护特性, 扩展了总线共模范围。RS-485 标准中, 数据信号采用差分传输方式, 最大传输距离为 1219m, 最大传输速率为 10Mbit/s。通常, RS-485 网络采用平衡双绞线为传输媒体, 平衡双绞线的长度与传输速率成反比, 只有输出速率在 20kbit/s 以下时, 才可能使用规定最长的电缆长度, 只有在很短的距离下才能获得最高传输速率。

一般来说, 15m 长双绞线的最大传输速率仅为 1Mbit/s。

TTL 电平到 RS-485 电平转换, 一般使用芯片 MC3487, RS-485 电平到 TTL 电平转换, 使用芯片 MC3486。MC3487 和 MC3486 的引脚如图 6-32 所示。

RS-485 总线应用与 MCS-51 单片机系统的数据传输时, 由单片机采集的数据经过电平转换, 转换为 RS-485 电平, 经过 RS-485 传输线传输, 接收端接收到 RS-485 电平, 再转换为 TTL 电平。

6.4.7 串行通信应用举例

1. 移位寄存器方式应用

串行口工作方式 0 为移位寄存方式, 常用于串口一并口的扩展, 通过方式 0 的发送方式完成串行一并行数据格式的转换, 通过方式 0 的接收完成并行一串行数据格式的转换。因此, 可以与具有并行输入串行输出、串行输入并行输出功能的芯片一起扩展并行口, 如前面所述 74LS164 芯片可以把串行输入的数据转换为并行输出, 74LS165 芯片可以把并行输入的数据转换为串行输出。下面举例说明这两种类型芯片和单片机串口的连接及程序控制应用。

【例 6-9】 串行口方式 0 输出扩展并行口。

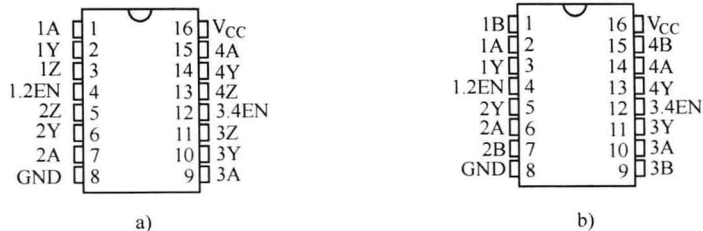


图 6-30 SN75174 和 SN75175 的引脚

a) SN75174 b) SN75175

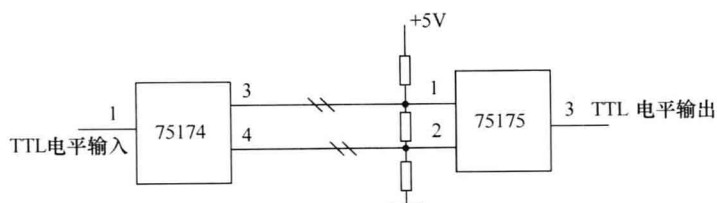


图 6-31 RS-422 接口电路

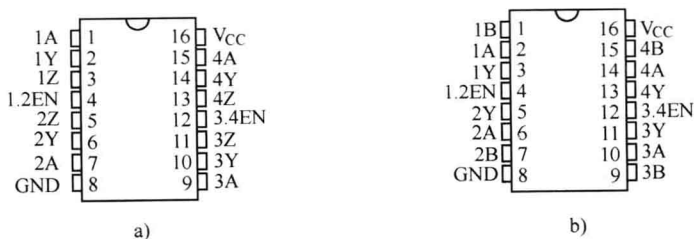


图 6-32 MC3487 和 MC3486 引脚

a) MC3487 b) MC3486

如图 6-33 所示, 串行口外接两片 74LS164, 编程使得 0#74LS164 的输出端并行输出存储单元 30H 单元数据, 1#74LS164 的输出端并行输出存储单元 31H 单元数据。74LS164 的 A、B 为串行输入端, $\overline{\text{MR}}$ 为清 0 控制端, $\text{Q}_0 \sim \text{Q}_7$ 为移位寄存器的输出, CP 为移位时钟脉冲信号输入端。当 $\overline{\text{MR}}$ 为高电平, CP 脉冲的上升沿到来时, $\text{Q}_0 = \text{A} \cdot \text{B}$, $\text{Q}_0 \sim \text{Q}_7$ 逐次右移 1 位, 当 $\overline{\text{MR}}$ 为低电平时, 输出端清 0。

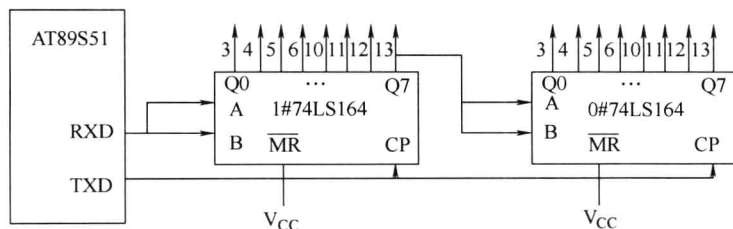


图 6-33 串行口扩展 16 位并行输出电路

	MOV	SCON,#00H	;设置串行口工作于方式 0
	MOV	R0,#30H	
	MOV	R2,#02H	
CIRCU:	MOV	A,@ R0	;先发送 30H 单元数据
	MOV	SBUF,A	;数据送入串口发送缓冲器,启动串口发送
WAIT:	JNB	TI, WAIT	;等待数据发送结束
	CLR	TI	
	INC	R0	
	DJNZ	R2,CIRCU	

C51 程序如下:

```
#include <reg52. h>
#define uchar unsigned char
#include <absacc. h>
#define data1 DBYTE[ 0x30 ]
#define data2 DBYTE[ 0x31 ]
void main ( void )
{
    SCON = 0;
    SBUF = data1;
    while ( TI = = 0 ) ;
    TI = 0;
    SBUF = data2;
}
```

【例 6-10】 串行口方式 0 并行输入口扩展。

如图 6-34 所示的 16 位接口电路读入数据,并把数据分别存放在 30H 和 31H 单元,其中 0#74LS165 的输入数据 D0 ~ D7 存放于 30H 单元,对应 D0 存放在 30H.0 位, D7 存放在 30H.7 位, 1#74LS165 的输入数据 D0 ~ D7 存放于 31H 单元。74LS165 是具有并行输入串行输出功能的 8 位移位寄存器, S/ $\bar{L}$ 为移位和并行数据装入控制, DS 为串行数据输入, QH 为串行数据输出, CLK 为时钟信号输入, CLK INH 为时钟禁止。S/ $\bar{L}$ = 0 时, 并行输入 A ~ H 的状态被锁入移

位寄存器,当 $S/\bar{L} = 1$, $CLK\ INH = 0$ 时,在移位时钟信号 CLK 的作用下,数据将由 QH 一位一位移位输出,先移出低端数据。

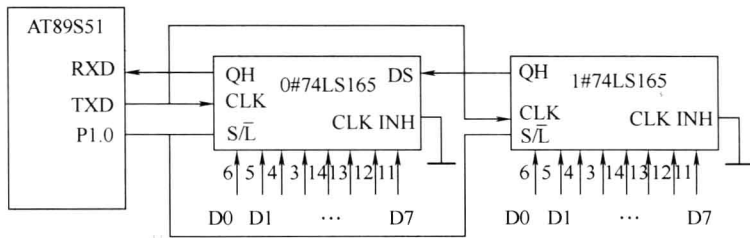


图 6-34 串行口扩展 16 位的并行输入电路

应用程序如下：

```

MOV R1, #02H      ;接收字节数
MOV R0, #30H      ;接收数据存储单元地址
CLR P1.0          ;并行输入数据锁入移位寄存器
NOP
SETB P1.0         ;允许移位寄存器移位工作
MOV SCON, #10H    ;设置串行口工作于方式 0,接收允许
WAIT: JNB RI, WAIT ;查询 RI 位是否为 1,等待接收数据
CLR RI           ;清除接收标志,准备下次接收
MOV A, SBUF
MOV @R0, A       ;存储接收数据
INC R0           ;修改存储单元地址
DJNZ R1, WAIT    ;2 字节数据是否接收完
RET

```

C51 程序如下：

```

#include <reg52.h>
#define uchar unsigned char
uchar i, data1[];
sbit P1_0 = P1^0;
void main (void)
{
    P1_0 = 0;
    nop();
    P1_0 = 1;
    SCON = 0x10;
    for(i = 0; i < 2; i++)
    {
        while(RI == 0);
        RI = 0;
        data1[i] = SBUF;
    }
}

```

说明：串行口和 74LS165 正常移位工作时，在移位脉冲的作用下，最先移入 RXD 端的是 0#74LS165 的数据 D0，下次移位脉冲上升沿到来时，移入 RXD 端的是 0#74LS165 的数据 D1，直至 16 次移位脉冲上升沿到来时，RXD 端的是 0#74LS165 的数据 D7，2 字节的数据全部移入串行口。

2. 单机、多机通信应用

串行通信方式通常用在传输数据距离较远的应用场合，但是需要说明的是，如果通信双方都采用 TTL 电平传送数据，其传输距离一般不超过 1.5m，若要提高串行通信的传输距离，需要采用其他接口形式，如 RS-232C、RS-485、RS-422 等。

AT89 系列单片机串行口有 3 种异步通信方式，最常用的是方式 1 和方式 3，它们的波特率是可变的，可以根据具体通信要求进行波特率设定。

串行口几种通信方式中，在软件设计时，都可以采用查询方式或者中断方式来实现，采用查询方式时，设置为不允许串行口中断，主程序中不断查询 RI 或者 TI 位是否为 1，若 RI、TI 为 0，说明串行口正在接收数据或者正在发送数据，则程序循环等待。若 RI = 1，说明一帧数据接收完，CPU 可以来把有效数据取走；若 TI = 1，则说明一帧数据发送完。采用中断方式，若 RI 或者 TI 为 0 时，正常执行主程序，则当 RI 或者 TI 为 1 时，转入串行口中断服务程序，执行串口中断程序。

【例 6-11】 两台单片机进行串行口通信，0#单片机发送数据，1#单片机接收数据，0#单片机将字符串 'AT89S52 Microcomputer' 发送到 1#单片机接收，并存储到其内部 RAM 从 30H 开始的存储单元。发送字符串以数据 0 结束，两个单片机的晶振频率均为 11.0592MHz，波特率均为 9600bit/s。试编写两个单片机的串口程序。

解：0#单片机和 1#单片机之间串行通信使用单工通信方式，一个单片机只发送数据，另外一个单片机只具有接收数据的功能。

1) 两个单片机应该工作于方式 1，8 位波特率可变的异步通信方式。

0#单片机工作于发送方式，SCON = 40H。

1#单片机工作于接收方式，SCON = 50H。

(2) 使用定时/计数器 T1 作为波特率发生器，T1 工作于方式 2，8 位自动重装初值的工作方式，波特率为 9600bit/s 时，查表可知计数初值应设定为 FDH。

0#单片机的发送程序如下：

```

MOV    TMOD, #20H           ;T1 工作于方式 2, 定时方式
MOV    TH1, #0FDH           ;计数初值为 FDH
MOV    TL1, #0FDH
MOV    SCON, #40H           ;串行口工作于方式 1
SETB   TR1                  ;启动定时/计数器 T1
SETB   TI                   ;为了便于用循环实现数据发送, 先置 TI
MOV    R4, #0                ;R4 作为字符串表指针

WAIT:  JNB   TI, WAIT
        CLR   TI
        MOV   DPTR, #TAB
        MOV   A, R4
        MOVC  A, @A + DPTR    ;取字符

```

```

JZ  RETURN                ;字符串以 0 结束
MOV  SBUF,A                ;发送字符
INC  R4                    ;为了取下一个字符,修改字符串指针
LJMP WAIT                  ;跳转到等待查询,是否发送完一个字符

RETURN: RET

TAB:DB  'AT89S52 Microcomputer' DB  0AH,0DH,0

```

C51 程序如下:

```

#include <reg52. h>
unsigned char string[] = {"AT89S52 Microcomputer\0"}; //字符串常量
unsigned char i=0;
main ( )
{
    TMOD = 0X20;
    TH1 = 0XFD;
    TL1 = 0XFD;
    SCON = 0X40;
    TR1 = 1;
    TI = 1;
    while( string[i]! =0)
    {
        while( TI = =0);
        TI = 0;
        SBUF = string[i];
        i + +;
    }
}

```

1#单片机的接收程序如下:

```

MOV  TMOD,#20H            ;T1 工作于方式 2,定时方式
MOV  TH1,#0FDH            ;计数初值为 FDH
MOV  TL1,#0FDH
MOV  SCON,#50H            ;串行口工作于方式 1,接收允许 REN = 1
SETB TR1                  ;启动定时/计数器 T1
MOV  R0,#30H              ;设置存储单元的地址指针
WAIT: JBC RI,NEXT          ;循环检测 RI 是否为 1,为 1,接收 SBUF 中的数据
      SJMP WAIT            ;不为 1,继续等待
NEXT: MOV  A,SBUF          ;取 SBUF 中接收的 1 字节的数据
      MOV  @R0,A            ;存放于存储单元
      INC  R0                ;修改单元地址指针,便于存放下一个数据
      LJMP WAIT
      RET

```

C51 程序如下:

```

#include <reg52. h>

```

```

unsigned char string[];    //字符串常量
unsigned char i=0;
main ( )
{
    TMOD = 0X20;
    TH1 = 0XFD;
    TL1 = 0XFD;
    SCON = 0X50;
    TR1 = 1;
    while(1)
    {
        while( RI == 0);
        RI = 0;
        string[i] = SBUF;
        i + + ;
    }
}

```

【例 6-12】 如图 6-35 所示，单片机系统使用 11.0592MHz 的晶体振荡器，串行通信的波特率为 4800bit/s，0#单片机可以向 1#单片机发送数据，1#单片机也可以向 0#单片机发送数据，两个单片机之间可以进行全双工串行通信。

解：(1) 分析

根据程序设计要求，0#单片机和 1#单片机地位相同，功能相似，任何一方都可以主动与对方通信，也可以响应对方的通信请求。为了实现双工通信，定义两种通信协议。

协议一 0#单片机发送/1#单片机接收

0#单片机需要向 1#单片机发送数据时，0#单片机先发出一个“接收数据请求”命令，1#单片机接收到命令后，如果准备好接收，则向 0#单片机发送“接收准备就绪”命令，0#单片机接收到这个响应命令后，开始向对方发送数据，直到通信结束，两个单片机返回初始状态。

协议二 0#单片机接收/1#单片机发送

0#单片机需要从 1#单片机接收数据时，0#单片机先发出一个“发送数据请求”命令，1#单片机接收到命令后，如果准备好发送，则向 0#单片机发送“发送准备就绪”命令，0#单片机接收到这个响应命令后，立即进入接收数据状态，直到通信结束，两个单片机返回初始状态。

程序中的几个命令，分别用 1 字节的数据表示如下：

“接收数据请求”命令——01H；

“接收准备就绪”命令——02H；

“发送数据请求”命令——10H；

“发送准备就绪”命令——20H。

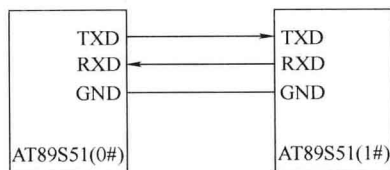


图 6-35 两个单片机串行通信

程序中设置两个标志位，位地址分别为 00H 和 01H。

00H——本机发送状态标志位。若为 0，表示本机发送命令状态；若为 1，表示本机发送数据状态。

01H——本机接收状态标志位。若为 0，表示本机接收命令状态；若为 1，表示本机接收数据状态。

(2) 程序

0#单片机的初始化程序：

```

ORG 0000H
LJMP MAIN
ORG 0023H
LJMP SINT
MAIN: ORG 0030H
MOV TMOD, #20H      ;T1 工作于方式 2, 定时方式
MOV PCON, #00H      ;SMOD = 0
MOV TH1, #0FAH      ;波特率为 4800bit/s 时, T1 的计数初值
MOV TL1, #0FAH      ;时间常数
SETB TR1            ;启动定时/计数器 T1
MOV IE, #90H         ;串行口中断允许
MOV SCON, #50H       ;串行口工作方式 1, 接收允许
CLR 00H              ;设置本机为发送命令状态
CLR 01H              ;设置本机为接收命令状态
MOV A, #01H          ;发“接收数据请求”命令
MOV SBUF, A
SJMP $

```

当 01H 命令发送完后，0#单片机置位 TI 标志位，转入中断服务程序执行，中断服务程序如下：

```

SINT:  JNB RI, SENT      ;非接收中断, 转去发送处理
        CLR RI
        JB 01H, R_DATA   ;位标志 01H 为 1, 表示为接收数据
        MOV A, SBUF       ;接收命令
        CJNE A, #01H, NEXT1 ;非“接收数据请求”命令
        MOV R6, #LEN_RCV  ;是“接收数据请求”命令, 准备接收数据
        MOV R1, #RCV_BUF  ;接收数据起始地址
        SETB 01H          ;设置本机为接收数据状态
        MOV A, #02H
        MOV SBUF, A        ;发送“接收准备就绪”命令
        LJMP RETURN
NEXT1:  CJNE A, #10H, NEXT2 ;非“发送数据请求”命令
        MOV R7, #LEN_TRN  ;是“发送数据请求”命令, 设置发送数据长度
        MOV R0, #TRN_BUF  ;设置发送数据起始单元地址
        SETB 01H          ;设置本机为发送数据状态
        MOV A, # 20H

```

```

        MOV  SBUF,A           ;发送“发送准备就绪”命令
NEXT2:  CJNE  A,#02H,NEXT3    ;对方“接收准备就绪”命令
        MOV  R7,#LEN_TRN      ;对方“接收准备就绪”命令,本机准备数据块发送,设置数
                                ;据块长度
        MOV  R0,#TRN_BUF      ;设置数据块起始地址
        SETB 00H              ;设置本机为发送数据状态
        MOV  A,@ R0           ;取数据
        MOV  SBUF,A           ;发送数据
        INC  R0               ;修改单元地址
        LJMP RETURN          ;跳转到中断返回
NEXT3:  CJNE  A,# 20H,NOCOM    ;对方“发送准备就绪”命令
        MOV  R6,#LEN_RCV      ;是对方“发送准备就绪”命令,本机准备接收数据
        MOV  R1,#RCV_BUF      ;设置接收数据起始地址
        SETB 01H              ;设置本机为接收数据状态
NOCOM:  LJMP  RETURN          ;跳转到中断返回
R_DATA: MOV  A,SBUF           ;取数据
        MOV  @ R1,A           ;存储数据
        INC  R1               ;修改存储单元地址
        DJNZ R6,RETURN        ;数据块是否接收完,R6 中为数据块长度
        CLR  01H              ;数据块接收完,置本机为接收命令状态
        LJMP RETURN
SENT:   CLR  TI               ;发送处理,清 TI 标志
        JB  00H,T_DATA        ;若位标志为 1,则为本机发送数据
        AJMP RETURN          ;本机发送命令
T_DATA: MOV  A,@ R0           ;取发送的数据
        MOV  SBUF,A           ;发送
        INC  R0               ;修改发送单元地址
        DJNZ R7, RETURN       ;数据块没发送完,直接中断返回,R7 中为数据块长度
        CLR  00H              ;数据块发送完,置本机为发送命令状态
RETURN:  RETI

```

C51 程序如下:

```

#include <reg52. h>

bit  bit0,bit1;                //定义两个位标志

unsigned char  a,i=0,j=0;

unsigned char  rcv_buf[ #LEN_RCV ],trn_buf[ #LEN_TRN];    //定义接收数组和发送数组

main ( )
{
    TMOD = 0X20;
    PCON = 0;
    TH1 = 0XFA;
    TL1 = 0XFA;
    TR1 = 1;
    IE = 0X90;
}

```

```

    SCON = 0X50;
    bit0 = bit1 = 0;
    SBUF = 0X01;                                     //发“接收数据请求”命令
    while(1);
}

uart ( ) interrupt 4 using 1                          //串口中断函数
{
    if (RI == 1)                                       //若为接收中断
    {
        RI = 0;
        if(bit1 == 1) {                               //判接收状态标志位,若为接收数据
            rcv_buf[j] = SBUF;                         //接收数据
            j++;
            if (j == #LEN_RCV) bit1 = 0;
        }
        else {                                         //若为接收命令,判断命令的类型
            a = SBUF;
            if (a == 1) { bit1 = 1;                    //接收到的为“接收数据请求”命令
                SBUF = 2;                               //发“接收准备就绪”命令
            }
            else if(a == 0x10) { bit1 = 1;             //接收到的为“发送数据请求”命令
                SBUF = 0X20;                             //发“发送准备就绪”命令
            }
            else if(a == 2) {                           //接收到的为“接收准备就绪”命令
                bit0 = 1;
                SBUF = trn_buf[i];                     //1#接收准备就绪,0#发送数据
                i++;
            }
            else if(a == 0x20) {                       //接收到的为“发送准备就绪”命令
                bit1 = 1;                               //使本机处于接收状态
            }
        }
    }
}

else { SBUF = trn_buf[i];                             //若为发送中断,发送数据
    i++;
    if (i == #LEN_TRN) bit0 = 0;
}
}

```

1#单片机的程序与 0#单片机程序完全一样,不再列出。

【例 6-13】 多机通信程序举例。

例 6-12 和例 6-13 两个例题中的串行通信,都是一个单片机和另一个单片机之间的通信,当单片机串行通信系统中的单片机有 3 个或者 3 个以上,并且其中一个为主单片机,其余的是从单片机,主从单片机的功能不同,这样的单片机应用系统为多机通信系统。

如前所述，串行口的控制寄存器 SCON 中的 SM2 为多机通信控制位。串行口以方式 2 或者方式 3 接收时，若 SM2 = 1 时，仅当接收到的第 9 位数据 RB8 为 1 时，数据才装入 SBUF 串口接收缓冲器，置位 RI；如果接收到的第 9 位数据 RB8 为 0 时，则不产生中断标志 RI，数据丢失。当 SM2 = 0 时，则接收到一个数据后，不管第 9 位数据 RB8 是何值，数据都装入 SBUF，并置位 RI。多机通信中就是应用单片机的这个特性，实现主从式多机通信。

如图 6-36 所示的多个单片机主从通信方式中，89S52 主单片机，连接了 3 个从单片机，主机控制与从机间的通信，从机的通信只能通过主机才能实现。系统中使用 TTL 电平通信，传输距离在 1.5m 内。

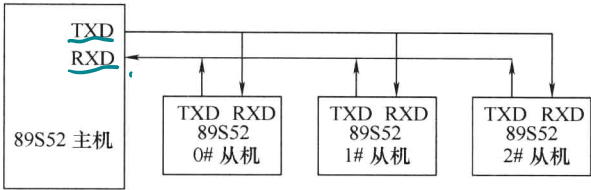


图 6-36 多机通信系统结构框图

主从机通信的方式如下：

- 1) 每个从机分配一个地址，占用 1 字节，3 个从机的地址分别为 0、1、2，各个从机的初始化程序将串行口编程为 9 位异步通信方式（方式 2 或者方式 3），置位多机通信标志 SM2 = 1，允许从机响应串行口接收中断。
- 2) 主机能够向特定的从机发送数据。主机给特定的某一从机发送数据。此时，其他从机不接收该数据，主机发送数据给从机时，先发送 1 字节的地址，以辨认与之通信的从机，主机发送地址信息的第 9 位数据为 1，地址信息所有的从机都能接收，主机发送的命令或数据信息的第 9 位数据为 0，只有地址信息与主机发送的地址信息一致的从机，才允许接收数据。
- 3) 从机接收到主机发送的地址信息，与自身的地址比较，如果一致，则清 SM2 = 0，如果不一致，仍然保持 SM2 = 1。
- 4) 从机接收主机发送的命令，命令信息第 9 位数据为 0，则 SM2 = 0 的从机正常接收该命令，SM2 = 1 的从机丢弃该命令。这样，可保证只有选中的从机才能有效接收命令。
- 5) 主机发送的命令为要求从机接收数据或者要求从机发送数据两种，根据命令情况从机作相应的处理程序：接收数据或者发送数据。
- 6) 从机接收完主机数据或向主机发送完数据后，重新置位 SM2 = 1，以便下次通信能够顺利实现。

如图 6-36 所示的多机通信系统中，系统使用 11.0592MHz 的晶振，波特率为 2400bit/s，主机发出的两种命令分别为 00H 和 01H，含义如下：

- 00H——要求从机接收数据；
- 01H——要求从机发送数据。

从机状态字字节地址为 20H，状态字定义如图 6-37 所示。

D7	D6	D5	D4	D3	D2	D1	D0
—	—	—	—	—	—	TRDY	RRDY

图 6-37 从机状态字

TRDY: TRDY = 1, 从机发送状态准备好; TRDY = 0, 从机发送状态未准备好。

RRDY: RRDY = 1, 从机接收状态准备好; RRDY = 0, 从机接收状态未准备好。

主机程序如下 (采用查询方式):

```
#include <reg52.h>

unsigned char  numbercj, numberjs, numberml, i;
unsigned char  bdata  state;

sbit  RRDY = state^0;
sbit  TRDY = state^1;

unsigned char  trndata[16] = {0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15};
unsigned char  revdata[16];

main ( )
{
    while(1) {
        TMOD = 0X20;
        PCON = 0;
        TH1 = 0XF4;
        TL1 = 0XF4;
        TR1 = 1;
        SCON = 0XD0;
        TB8 = 1;
        SBUF = numbercj;           //发送从机地址
        while( TI == 0 );
        TI = 0;
        while( RI == 0 );
        RI = 0;
        numberjs = SBUF;
        if( numberjs == numbercj ) {
            TB8 = 0;
            numberml = SBUF;       // 发送命令
            while( TI == 0 );
            TI = 0;
            while( RI == 0 );      //等待从机状态反馈
            RI = 0;
            state = SBUF;
            if ( numberml == 0 ) {
                if( RRDY == 1 ) {
                    for( i = 1; i <= 16; i + + ) {
                        TB8 = 0;
                        SBUF = trndata[ i ];
                        while( TI == 0 );
                        TI = 0;
                    }
                }
            }
        }
    }
}
```

```

    }
    else {
        if(TRDY == 1) {
            for(i = 1; i <= 16; i++) {
                while (RI == 0);
                RI = 0;
                trndata[i] = SBUF;
            }
        }
    }
}
}
}

```

从机程序如下(采用中断方式):

```

#include <reg52.h>
unsigned char  numbercj,address,numberml,i;
unsigned char  bdata state;

sbit  RRDY = state^0;
sbit  TRDY = state^1;

unsigned char  trndata[16] = {0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15};
unsigned char  revdata[16];
main ( )
{
    TMOD = 0X20;
    PCON = 0;
    TH1 = 0XF4;
    TL1 = 0XF4;
    TR1 = 1;
    SCON = 0XF0;
    while(1);
}

uart ( ) interrupt 4 using 1
{
    if (RI == 1)
    {
        RI = 0;
        if (RB8 == 1) { numbercj = SBUF;
                        if(numbercj == address)
                        { SBUF = numbercj          ;//反馈从机地址
                          SM2 = 0;
                          state = 0; }
                    }
        else { if (RRDY == 0) {
                    numberml = SBUF;
                    if(numberml == 0) {

```

```

        RRDY = 1;
        TRDY = 0;
        SBUF = state;
    }
    else { TRDY = 1;
        RRDY = 0;
        SBUF = state;
    }
}

else { for( i = 1; i <= 16; i++ ) {
    while ( RI == 0 );
    RI = 0;
    trndata[ i ] = SBUF;
}
    SM2 = 1;
    RRDY = 0;
}
}

else { TI = 0;
    if ( TRDY == 1 ) {
        for( i = 1; i <= 16; i++ ) {
            TB8 = 0;
            SBUF = trndata[ i ];
            while( TI == 0 );
            TI = 0;
        }
        SM2 = 1;
        TRDY = 0;
    }
}
}
}

```

习 题 六

1. 简述 AT89S 系列单片机的并行口资源及其功能。
2. 设计一个声光报警器，设备正常运行时，绿色指示灯亮，若设备非正常运行时，红灯闪烁、报警器持续发声报警。
3. 简述 AT89S 系列单片机内部中断源，并指出各中断源中断服务程序入口地址。
4. 设计一个外部事件中断计数器，使用外中断 0 的边沿触发方式，对外部发生的中断事件进行计数。
5. 根据计数器结构不同，定时/计数器 T0、T1 分别有哪些工作方式？根据计数脉冲不同，T0、T1 有哪些工作方式？
6. 单片机系统使用 12MHz 的晶振，使用定时/计数器 T0 产生 250μs 的定时，使 P3.4 输出周期为 1s 的方波，试编写主程序和 T0 中断服务程序。
7. 单片机系统使用 12MHz 的晶振，使用定时/计数器 T2 产生 50ms 的定时，每隔 1s 时钟单元 30H ~ 32H（时、分、秒）计时，试编写主程序和 T2 中断服务程序。

8. 简述 AT89S 系列单片机有哪几种串行口工作方式。

9. 简述 AT89S 系列单片机多机通信原理。

10. 单片机系统晶振为 11.0592MHz，串行口工作于方式 1，波特率为 4800bit/s，从串行口输出字符“AT89S52 Micro computer”。试分别用查询方式和中断方式编写程序。

11. 0# 单片机以波特率 1200bit/s，从串行口发送内部 RAM20H ~ 30H 单元的数据块，晶振为 11.0592MHz，1# 单片机从串行口接收数据，并将数据保存于其内部存储器中，试编写串口通信程序。